AI Gateway — Verified Technique Inventory

55 techniques across 7 areas · adversarially fact-checked against primary sources

ARVAN CLOUD AI GATEWAY · SHARIF UNIVERSITY · DESIGN PHASE · GENERATED 2026-07-24

AI Gateway — Verified Technique Inventory

Full per-technique research inventory produced by 7 parallel research agents and corrected by 7 adversarial fact-check agents (primary-source verification). Labels: **evidence-based** = number located in the named paper/benchmark; **reasoned-estimate** = derived from adjacent evidence (stated); **unverified-assumption** = needs measurement.

Predictive model routing (one-shot pre-dispatch model selection)

Config-rule / heuristic tiered routing (cheap-balanced-quality policy rules)

- **Problem it solves:** Assigns each request to a model tier using declarative rules (intent class, prompt length, tenant policy, budget headroom) with zero training data and near-zero decision latency.
- **Pipeline stage:** routing-decision
- **Quality impact:** -2..-10% vs always-strongest-model on mixed traffic (misroute tail on hard requests that look easy); +5..+25% vs always-cheapest on hard requests. Holds only when rules are conservative and traffic skews toward easy requests; no per-query difficulty signal, so the tail is unbounded on adversarially odd traffic.
- **Latency impact:** Router overhead <1ms (rule/CEL evaluation) — the only option that trivially meets the ms-scale overhead requirement 4.2.1. End-to-end latency typically -20..-50% vs all-strong dispatch because most traffic lands on small fast self-hosted models.
- **Cost impact:** -30..-60% vs all-strong-model dispatch on typical skewed-easy traffic; extrapolated from RouteLLM (-45..-85%, verified but benchmark-specific best-case operating points vs all-GPT-4 dispatch) and FrugalGPT (up to -98%, verified but explicitly the best-case cascade figure on its benchmarks) as the no-ML lower bound — neither figure transfers 1:1 to production traffic. Zero training or serving cost for the router itself.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Phase-1 mandate in the project's own strategy brief (AIGateway__4_.md): no labeled data, no GPU, <1ms overhead, fully expressible in the YAML/GitOps policy config the requirements doc demands.
- **Sources:**
- RouterBench (Hu et al. 2024) — simple/non-predictive baselines competitive with learned routers on several tasks — https://arxiv.org/abs/2403.12031
- FrugalGPT (Chen, Zaharia, Zou 2023) — https://arxiv.org/abs/2305.05176
- RouteLLM (Ong et al. 2024) — learned-router numbers used as upper bound for extrapolation — https://arxiv.org/abs/2406.18665

Preference-aligned intent routing with a small causal LM (Arch-Router-1.5B on archgw fork)

- **Problem it solves:** Maps each request to a human-authored routing policy (domain + action) via a 1.5B generative router, decoupling 'what the request is' from 'which model serves it' so model swaps and additions are config-only, no retraining.
- **Pipeline stage:** routing-decision
- **Quality impact:** 93.17% routing-decision accuracy overall (98.11 on fine-grained query-to-policy matching), beating top proprietary LLM routers by avg +7.71% (verified verbatim in the paper — but measured on the paper's own curated conversational benchmark with well-designed policies, not production traffic); the ~7% misroute rate translates to roughly -2..-7% end-answer quality vs oracle policy routing (that translation is a reasoned estimate — the paper measures routing accuracy, not downstream answer quality). Garbage policies = garbage routing.
- **Latency impact:** 51±12ms router overhead per request (paper's measured latency, GPU-served); paper states over 28x faster than the closest competitor (Claude-Sonnet-3.7 at 1450±385ms). This is the ceiling of the project's

'milliseconds-scale' requirement — negligible vs seconds-scale LLM generation but the heaviest router option here; needs a small dedicated GPU slice (~3GB fp16, less quantized) to hit that number.

- **Cost impact:** Router serving cost small but nonzero (1.5B model on shared H100 capacity). Enables the same tiering savings as rule routing (-30..-60%, traffic-dependent, reasoned) with fewer misroutes; no retraining cost when models/policies change — new routes are added in-context at inference time.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — The strategy brief explicitly commits to forking archgw with Arch-Router-1.5B in Phase 1; policy-name outputs double as the human-readable XAI explanation (req 4.1.4), and config-only route addition partially covers the new-model onboarding requirement without any meta-learning machinery.
- **Sources:**
- Arch-Router: Aligning LLM Routing with Human Preferences (Tran et al. 2025) — https://arxiv.org/abs/2506.16655
- archgw (katanemo) — the engine the project plans to fork — https://github.com/katanemo/archgw
- VentureBeat summary (93% accuracy, no retraining) — https://venturebeat.com/technology/new-1-5b-router-model-achieves-93-accuracy-without-costly-retraining

Preference-trained win-rate routers (RouteLLM: matrix-factorization vs BERT vs causal-LM predictors)

- **Problem it solves:** Learns a calibrated probability that the strong model's answer beats the weak model's for a given query, exposing a single cost threshold that dials the cost-quality trade-off per request.
- **Pipeline stage:** routing-decision
- **Quality impact:** Retains 95% of GPT-4-level quality at the reported operating points (verified against the LMSYS release blog; these are benchmark-specific best-case operating points — the paper abstract itself claims only 'over 2x' cost reduction); on RouteLLM's benchmarks routers generalize to model pairs not seen in training. Caveat from the DSC fragility study: the BERT variant over-relies on surface domain cues (routes ALL math/coding to the strong model). Holds on MT-Bench/MMLU/GSM8K-like traffic with Chatbot-Arena-style preference training data plus augmentation.
- **Latency impact:** Matrix-factorization scoring is microsecond-to-sub-ms (cited as the best accuracy/overhead balance); BERT ~5-20ms; causal-LM router 20-100ms (encoder/decoder-scale inference — reasoned split, the paper reports architectures, not per-variant latency). Net end-to-end latency drops when most traffic routes to the weak/fast model.
- **Cost impact:** Over -85% on MT-Bench, -45% on MMLU, -35% on GSM8K vs all-GPT-4 dispatch while holding 95% quality (verified LMSYS blog/paper-experiment numbers — best-case benchmark operating points computed against all-strong dispatch, unlikely to transfer 1:1 to production traffic); also measured >40% cheaper than commercial router offerings (Martian, Unify) at equal performance, MT-Bench only. Training cost near-zero for the MF variant (public Arena preference data, small model).
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — The project's own Tier-3 decision defers trained routers to Phase 4 (gate: rules leave >15-20% savings uncaptured) — correct call; but note the MF variant is nearly free to train on public Arena data, is microsecond-scale, and generalizes to unseen model pairs, so it is the cheapest first trained router when that gate trips.
- **Sources:**
- RouteLLM: Learning to Route LLMs with Preference Data (Ong et al., ICLR 2025) — https://arxiv.org/abs/2406.18665
- LMSYS RouteLLM release blog — headline percentages live here, not in the paper abstract — https://www.lmsys.org/blog/2024-07-01-routellm/
- lm-sys/RouteLLM framework — https://github.com/lm-sys/routellm

Per-request quality/cost/latency prediction as the routing substrate (quality-gap meta-model + deterministic cost model + observed-latency model)

- **Problem it solves:** Implements requirements ch.4.1.1 directly: predict all three indicators (answer quality, inference cost, latency) per (prompt, candidate-model) pair, then filter candidates by budget and argmax predicted quality.
- **Pipeline stage:** routing-decision

- **Quality impact:** Neutral by design at the chosen operating point: Hybrid LLM's quality-gap router achieves up to 40% fewer calls to the large model with no measured drop in response quality (verified verbatim in the arXiv abstract; 'up to' marks the best-case router/threshold configuration, and quality neutrality is judged by BART-score-derived labels on the paper's test distribution, not production traffic). Degrades when the quality predictor is miscalibrated on out-of-distribution traffic — calibration monitoring is mandatory.
- **Latency impact:** Quality predictor adds ~1-10ms (BERT/DeBERTa-scale encoder — reasoned from model size); cost prediction is near-deterministic bookkeeping (price-per-token x predicted output length) at ~0ms; latency prediction from per-model observed p50/p95 plus output-length estimate is near-free but individually noisy (per-request output-length error commonly +/-20-40%, better in aggregate — reasoned from response-length-perception literature).
- **Cost impact:** -20..-40% via avoided strong-model calls (scaled down from the Hybrid LLM up-to-40%-fewer-calls best case by traffic mix); the prediction infrastructure itself costs almost nothing to serve. Also the only substrate that can enforce 'never knowingly exceed the budget' BEFORE dispatch rather than after.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Ch.4.1.1 makes tri-metric prediction a hard requirement, and two of the three predictors (cost, latency) are deterministic bookkeeping buildable in Phase 1 from vLLM telemetry — only the quality predictor needs data and can wait for Phase 4.
- **Sources:**
- Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing (Ding et al., ICLR 2024) — arXiv version (verified) — https://arxiv.org/abs/2404.14618
- Hybrid LLM OpenReview record (bot-gated to automated fetch) — https://openreview.net/forum?id=02f3mUtqnM
- Fly-Swat or Cannon? Cost-Effective Language Model Choice via Meta-Modeling (Sakota et al., WSDM 2024) — https://arxiv.org/abs/2308.06077
- Microsoft Research publication page — https://www.microsoft.com/en-us/research/publication/hybrid-llm-cost-efficient-and-quality-aware-query-routing/

Few-shot / graph-based cold-start routing for newly added models (GraphRouter; UniRoute probe-set fingerprinting)

- **Problem it solves:** Routes to a newly onboarded model with few or zero logged interactions by representing models as feature vectors (graph-edge prediction over task/query/LLM nodes, or prediction-vectors on a fixed probe set) instead of per-model trained heads.
- **Pipeline stage:** routing-decision
- **Quality impact:** GraphRouter: minimum +12.28% on its Reward metric (joint quality-cost objective; verified — paper body states 'minimum effect improvement of 12.28% on metric Reward', abstract rounds to 12.3%) over strongest prior routers on its four-task benchmark, and sustains performance on NEW LLM nodes given only few-shot data, no full retrain (at least +9.5% in the new-LLM/low-resource setting). UniRoute: effective routing across 30+ LLMs entirely unseen at training time (verified in abstract); its headline contribution is theoretical (excess risk bound), so treat baseline-beating magnitudes as indicative, not portable. Both are academic benchmarks, not production traffic.
- **Latency impact:** GNN or cluster-map forward pass is ~1-5ms at inference (small network — reasoned from architecture); the real cost is offline: onboarding a model requires running it once over a representative probe set (minutes-to-hours of batch inference, off the critical path).
- **Cost impact:** Onboarding cost collapses to probe-set inference on self-hosted GPUs (near-free) instead of weeks of live suboptimal routing while logs accumulate; new cheap models start absorbing traffic immediately, which is where their cost saving actually comes from.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Few-shot onboarding of new models is an explicit, differentiating research axis of THIS project (reqs 4.1.3 and 5.5), and the UniRoute probe-set idea degrades gracefully into the pragmatic near-term version: Arch-Router config-only routes plus promptfoo golden-set profiling of every new model.
- **Sources:**
- GraphRouter: A Graph-based Router for LLM Selections (Feng et al., ICLR 2025) — https://arxiv.org/abs/2410.03834

- Universal Model Routing for Efficient LLM Inference (Jitkrittum et al. 2025) — https://arxiv.org/abs/2502.08773
- GraphRouter project page — https://ulab-uiuc.github.io/GraphRouter/

Router robustness hardening (paraphrase-consistency audits, category-bias checks, adversarial-rerouting defenses, OOD fallback-to-default)

- **Problem it solves:** Protects the routing decision itself: paraphrases and surface cues flip router decisions, and query-independent adversarial suffixes can force traffic to the expensive tier, silently breaking the per-request budget guarantee.
- **Pipeline stage:** routing-decision
- **Quality impact:** Protective rather than additive: the DSC study (verified verbatim in abstract; venue EACL 2026 confirmed via ACL Anthology) shows preference routers make category-driven decisions — a BERT router directed ALL coding/math queries to the most powerful LLM even when unneeded, and routed jailbreak attempts to WEAKER models, a safety leak; hardening claws back the misrouting tail, which I estimate at -5..-15% quality on affected paraphrased/OOD slices if left undefended (reasoned — the paper demonstrates the failure mode, not an aggregate quality delta).
- **Latency impact:** +0ms if done as offline audits (paraphrase-perturbed golden set replayed in dry-run mode); +1-51ms if an online consistency check re-scores a perturbed copy of the prompt (a second router pass at whatever the router's own latency is). Input sanitization against gadget suffixes is <1ms.
- **Cost impact:** Blocks adversarial cost inflation: confounder-gadget attacks succeed in both white-box and black-box settings against open-source AND commercial routers, letting an attacker push essentially all their traffic to the expensive tier while response quality is unaffected (so it goes unnoticed); without defense the cost exposure is unbounded per hostile tenant. Defense cost itself is negligible.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — The product's entire promise is 'never knowingly exceed the budget' in a multi-tenant cloud gateway — a routable-by-attacker router voids that promise, and the already-planned promptfoo + dry-run infrastructure makes paraphrase/bias audits nearly free to run.
- **Sources:**
- How Robust Are Router-LLMs? (Kassem, Scholkopf, Jin, EACL 2026; DSC benchmark) — https://arxiv.org/abs/2504.07113
- Rerouting LLM Routers (Shafran, Schuster, Ristenpart, Shmatikov, COLM 2025) — https://arxiv.org/abs/2501.01818
- ACL Anthology version — https://aclanthology.org/2026.eacl-long.351/

Offline routing simulation on precomputed outcomes (RouterBench methodology powering dry-run mode and threshold calibration)

- **Problem it solves:** Evaluates and calibrates any routing policy or threshold by table lookup over precomputed (prompt, model, answer, quality, cost, latency) tuples, so policy changes are tested without live inference cost or user-traffic risk.
- **Pipeline stage:** offline
- **Quality impact:** Indirect but decisive: it is how the operating point on the cost-quality curve is chosen (AIQ / convex-hull analysis). Key RouterBench findings to internalize (verified in full text): cascading routers can EXCEED a single high-end model's quality at lower cumulative cost — but only when the cascade's quality judge has an error rate <=0.1, and performance 'may deteriorate rapidly' once judge error exceeds 0.2 (cascades also pay for multiple inference calls); meanwhile the paper is blunt about learned predictive routers: in its comparative analysis none of the tested routing algorithms significantly outperformed the Zero-router baseline — which independently validates this project's rules-and-cascade-first, trained-router-later ordering.
- **Latency impact:** 0ms at runtime — entirely off the serving path; a full policy sweep over a 405k-outcome table runs in seconds-to-minutes on CPU.
- **Cost impact:** Near-zero marginal evaluation cost after a one-time precompute of the golden set (200-500 requests per the strategy brief) across all self-hosted models — cheap on owned H100s; eliminates paid live A/B experiments for every threshold change, and RouterBench's own 405k-outcome dataset is free bootstrap data before any local traffic exists.

- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Dry-run mode is a hard functional requirement (1.1.4) and promptfoo is already the planned harness — extending it with a precomputed-outcome table gives threshold calibration, router regression tests, and the Win-Rate/Cost-Savings logging (4.1.5) from one artifact.
- **Sources:**
- RouterBench: A Benchmark for Multi-LLM Routing System (Hu et al. 2024) — 405k+ inference outcomes, 11 LLMs (+3 RAG-specific), 8 datasets spanning 64 tasks plus 1 RAG dataset — https://arxiv.org/abs/2403.12031
- Martian RouterBench announcement — https://withmartian.com/post/introducing-routerbench
- OpenReview record — https://openreview.net/forum?id=IVXmV8Uxwh

Fact-check verdicts for this area

Claim	Verdict	Note
Arch-Router: 93.17% overall routing accuracy, +7.71% avg over top proprietary routers, 98.11 fine-grained matching, 51ms latency, >28x faster than closest compe	**confirmed**	All five numbers verified verbatim in the arXiv full text: 'highest overall routing score of 93.17%, surpassing every other candidate model on average by 7.71%'; fQfA 98.11; Table 9 latency 51±12ms vs Claude-Sonnet-3.7 at 1450±385ms ('over 28 times faster'). Caveat: measured on the paper's own curat
RouteLLM: cost reductions -85% MT-Bench / -45% MMLU / -35% GSM8K while holding 95% of GPT-4 quality; >40% cheaper than commercial routers	**confirmed**	Matches the LMSYS release blog: 'cost reductions of over 85% on MT Bench, 45% on MMLU, and 35% on GSM8K... while still achieving 95% of GPT-4's performance' and 'same performance as these commercial routers while being over 40% cheaper' (vs Martian and Unify, MT-Bench only). Nuance: the paper abstra
Hybrid LLM: up to 40% fewer calls to the large model with no drop in response quality	**confirmed**	Exact abstract wording verified on the arXiv version (2404.14618; the cited OpenReview page is bot-gated and unverifiable by fetch): 'our approach allows us to make up to 40% fewer calls to the large model, with no drop in response quality.' Caveat: 'up to' marks the best-case router/threshold confi
GraphRouter: minimum +12.28% reward over strongest prior routers; UniRoute routes across 30+ LLMs unseen at training time	**confirmed**	GraphRouter paper body: 'delivering a minimum effect improvement of 12.28% on metric Reward' (abstract rounds to 12.3%); the analyst's 'reward (joint quality-cost objective)' attribution is correct. UniRoute abstract confirms 'routing amongst more than 30 unseen LLMs', though its headline contributi
RouterBench: 405k+ inference outcomes, 11 LLMs, 7+ tasks; cascading routers can EXCEED a single high-end model's quality at lower cost while predictive routers	**corrected**	405k+ outcomes and 11 LLMs (6 open, 5 proprietary, +3 RAG-specific) confirmed; '7+ tasks' is wrong — it is 8 datasets spanning 64 tasks plus 1 RAG dataset. The cascade-exceeds-single-model finding is real but conditional and the findings omitted the condition: it holds when the cascade's quality jud
DSC robustness study (EACL 2026): a BERT-based preference router sends ALL coding/math queries to the strongest LLM and routes jailbreak attempts to weaker mode	**confirmed**	Both failure modes appear verbatim in the abstract: 'a BERT-based router directs all coding and mathematics queries to the most powerful LLM even when simpler models would suffice, while routing jailbreaking attempts to weaker models, thereby elevating safety risks.' Venue EACL 2026 confirmed via AC

Cascading / fallback chains + answer quality estimators

FrugalGPT-style scorer-gated sequential cascade (cheap model first, escalate on low score)

- **Problem it solves:** Paying strong-model price and latency on every request when the majority of traffic is answerable by a cheap self-hosted model.
- **Pipeline stage:** execution
- **Quality impact:** -5..+4% vs always-strongest-model baseline. Best case +1.5..+4% (paper: 'improve the accuracy over GPT-4 by 4% with the same cost'; HEADLINES case study: +1.5% accuracy at -80% cost, Fig. 3c — a single-dataset case-study figure). The +% cases require a well-tuned per-task DistilBERT scorer on classification-like tasks (HEADLINES/OVERRULING/CoQA); on open-ended chat expect 0..-5% regression, which is exactly the project's Phase 2 gate (<5% regression on golden set).
- **Latency impact:** Bimodal: non-escalated majority (60-90% of traffic) -30..-70% TTFT+total (small model decodes faster, no strong-model queue); escalated tail +40..+110% vs routing straight to the strong model, because the failed cheap generation + scoring + strong generation stack serially. Approaches ~2x when cheap-tier output is

long (CoT) or scoring itself is a model call. This hidden tail penalty is why time-budgeted requests need the deadline guard (technique 8). (Latency figures are analyst estimates, not paper-reported.)

- **Cost impact:** -98% is the paper's best case (HEADLINES: 98.3% savings at matched quality, Table 3) on skewed easy-heavy benchmark traffic; other tasks in the same table save 59-73%. Realistic on mixed production chat traffic -30..-60% (project brief itself targets 50%+). Savings scale with the easy-traffic fraction and scorer precision; a scorer that over-escalates erases savings, one that under-escalates ships garbage. Marginal note: with self-hosted models 'cost' = GPU-seconds freed, which converts to throughput headroom on the H100s.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — It IS the project's chosen backbone (Tier 1, Phase 2): needs no labeled router dataset, degrades gracefully, and maps directly onto the per-request time/cost budget dial; escalation targets are self-hosted so no dependency on unreliable closed APIs.
- **Sources:**
- FrugalGPT (Chen, Zaharia, Zou; arXiv 2023, published TMLR 12/2024) https://arxiv.org/abs/2305.05176
- Project strategy brief 'AIGateway (4).docx' §3 (adopts FrugalGPT as backbone, flags 98% as best-case, targets 50%+ cost cut) — verified in doc

Abstention-aware / early-abstention cascade (cheap model prompted or tuned to say 'cannot answer' as the escalation trigger)

- **Problem it solves:** A small model generating a full confident-but-wrong answer wastes tokens and latency before the scorer can trigger escalation; letting it abstain early anticipates the deferral decision.
- **Pipeline stage:** execution
- **Quality impact:** Error rate -5.0% and overall test loss -2.2% (averages across GSM8K, MedMCQA, MMLU, TriviaQA, TruthfulQA, XSum — per-benchmark results vary), bought with a +4.1% increase in overall abstention rate; holds when small- and large-model error patterns are correlated, which the paper shows they are. Caveat: 2025 arXiv preprint, not yet peer-reviewed, benchmark traffic only — production transfer unproven.
- **Latency impact:** On abstained requests, escalation fires after a short abstention emission instead of a full generation + scoring pass: reasoned estimate -20..-50% of the cascade tail penalty on those requests (abstention output is tokens-short vs a full answer). Neutral on non-abstained traffic.
- **Cost impact:** -13.0% total cascade cost vs a standard (final-model-only abstention) cascade, same benchmark suite (preprint benchmark average — treat as upper-bound guidance, not a production forecast); prompt-level abstention costs ~0 to implement, fine-tuned calibrated abstention costs one offline training run on owned H100s.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Prompt-level abstention is near-free on self-hosted Qwen/Llama and directly attacks the cascade's two weak points (wasted cheap-tier tokens and tail latency); no external dependency, no labeled data required for the prompted variant.
- **Sources:**
- Cost-Saving LLM Cascades with Early Abstention (2025, arXiv preprint) https://arxiv.org/abs/2502.09054

Unified cascade routing (Dekoninck et al.: routing and cascading as one budget-constrained policy)

- **Problem it solves:** A fixed choice between one-shot routing and sequential cascading is provably suboptimal when query difficulty is only partly observable before generation; the unified policy re-decides after each model output using both pre- and post-generation signals.
- **Pipeline stage:** routing-decision
- **Quality impact:** +1..4% absolute quality (AUC over cost-quality curve) vs the best of pure routing or pure cascading on RouterBench (3-11 models, controlled estimator noise; gap widens with more models, narrows at higher noise). On realistic benchmarks the margin tracks estimator quality: up to 14% on SWE-Bench where accurate post-hoc quality estimates exist, shrinking to <=1.2% when quality estimates are noisy — the paper is explicit that noisy quality estimators are the binding constraint. Key theory result for this project: routing wins when difficulty is discernible from the prompt (and skips the cascade tail-latency penalty entirely); cascading wins when difficulty is ambiguous pre-generation because token-level output signals are informative.

- **Latency impact:** The paper publishes no per-query ms figure; its Appendix F.2 runtime study (random data, log-scale figure) shows the greedy variant stays computationally trivial even at 80 models while the standard variant reaches ~1s at 40 models. For this project's 2-5 model tiers, low-single-digit-ms per decision is a plausible read-off but is an estimate, not a published number — verify on own hardware before trusting the ms-scale overhead budget. Routing obviously-hard queries straight to the strong tier removes their +40..110% escalation penalty.
- **Cost impact:** At matched quality, equal or lower cost than either pure strategy (dominates the Pareto frontier on RouterBench); no extra inference cost, it reuses the same scorers.
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — The correct end-state architecture and its overhead plausibly fits the ms constraint at 2-5 models (unverified in the paper), but its edge over plain cascading collapses without calibrated quality/cost estimators the team won't have until logged traffic exists — matches the strategy brief's Phase 4 'trained router only if rules leave >15-20% uncaptured'.
- **Sources:**
- A Unified Approach to Routing and Cascading for LLMs (Dekoninck, Baader, Vechev; ICML 2025 — earlier version at ICLR 2025 SCOPE workshop) https://arxiv.org/abs/2410.10347
- Code: https://github.com/eth-sri/cascade-routing

Deterministic tier-0 verifiers as cascade scorers (JSON-schema validation, code execution/compile, regex/format/length checks)

- **Problem it solves:** Judging answer quality normally costs tokens and latency, but on structured traffic (code, JSON, extraction, math-with-answer-format) correctness is mechanically checkable for free with near-zero false accepts.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** On checkable traffic, escalation-trigger precision approaches 100% for the checked property (invalid JSON / failing tests never get waved through), which is what makes best-of-N and refine loops pay off at all; % end-quality lift is traffic-mix dependent — reasoned estimate +5..20% task-success on structured-output/code slices vs no-verifier cascade, extrapolated from execution-filtering results in code generation and promptfoo-style assertion practice. No effect on unverifiable prose.
- **Latency impact:** +0..5ms for regex/schema/length (negligible, in-process); +100ms..2s for sandboxed code execution, bounded by a hard timeout — only spent on the code slice and still far cheaper than a judge-model call.
- **Cost impact:** ~0 model tokens (pure CPU); enables cost-free rejection of bad cheap-tier answers, so it raises cascade savings rather than costing anything.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Free, deterministic, millisecond-scale, zero external dependency — and the project's own engineering principles ('the expensive end of the dial only pays off with a checkable signal') make this the mandatory first scorer layer.
- **Sources:**
- FrugalGPT scoring heuristics discussion https://arxiv.org/abs/2305.05176
- promptfoo deterministic assertions (project's chosen eval harness) https://www.promptfoo.dev/docs/configuration/expected-outputs/
- Project brief 'AIGateway (4).docx' ('Never refine without a verifier' — verified in doc)

Logprob/confidence-based deferral (Chow's rule on softmax confidence; mean and quantile token logprobs of the generated sequence)

- **Problem it solves:** The cascade needs an is-this-answer-good-enough signal without paying for any extra model call; token logprobs are a free byproduct of decoding on self-hosted vLLM.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** In-distribution, confidence-based deferral 'often works remarkably well in practice' (NeurIPS 2023, verbatim) — the ~1-2%-of-oracle-deferral gap is an analyst reading of their classification-suite plots, not a paper-stated number; it degrades in exactly three characterized settings: specialist downstream models, label noise, and distribution shift. For generative outputs, plain mean sequence logprob is length-biased; quantile-based features +

a small post-hoc deferral rule beat it across FLAN-T5 benchmark suites (consistent cost-quality curve dominance, task-dependent magnitude).

- **Latency impact:** ~0ms — logprobs come free with each decoded token in vLLM; the threshold comparison is microseconds. The only scorer class with strictly zero added latency.
- **Cost impact:** 0 extra tokens/compute; recalibration effort (not runtime cost) each time a tier model is swapped, since confidence scales are not comparable across models.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Uniquely suited here: self-hosted vLLM exposes full logprobs (closed APIs often don't — and they're unreliable in this market anyway), it costs zero dollars and zero milliseconds, satisfying both the near-zero budget and ms-overhead constraints; use quantile features, not raw mean logprob, and recalibrate per model.
- **Sources:**
- When Does Confidence-Based Cascade Deferral Suffice? (Jitkrittum et al., NeurIPS 2023) https://arxiv.org/abs/2307.02764
- Language Model Cascades: Token-level Uncertainty and Beyond (Gupta et al., 2024) https://arxiv.org/abs/2404.10136

Small trained quality estimator (DistilBERT-scale head scoring query+answer pairs)

- **Problem it solves:** Deterministic checks can't judge open-ended answer quality and logprobs are blind to factual wrongness stated confidently; a tiny trained scorer gives a learned quality score in milliseconds.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** This is the scorer behind FrugalGPT's headline numbers (paper verbatim: 'We employ a DistilBERT tailored to regression as the scoring function') — it is what lets the cascade retain ~95-100% of strong-model quality at -59..-98% per-task cost (Table 3 range), conditions: trained per task family on labeled (query, answer, correct?) data, evaluated on classification-like benchmarks; BERT-scale scorers/routers are known to over-rely on surface features and misjudge out-of-domain traffic (RouteLLM-line analyses), so expect degradation on open-domain chat until retrained on logged traffic.
- **Latency impact:** +5..30ms per scored answer (~66M params, GPU co-located; ~50-100ms CPU-only) — reasoned estimate from model size, not a benchmarked figure; comfortably within ms-scale overhead if GPU-co-hosted next to the vLLM pods.
- **Cost impact:** Near-zero marginal serving cost (a sliver of one GPU); real cost is data: needs roughly 500-5k labeled examples per task family to train — bootstrappable from the promptfoo golden set plus judge-labeled production logs.
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — High payoff but blocked on labeled data the team doesn't have on day one — correctly a Phase 3/4 item: start with verifiers+logprobs, log traffic, label with the offline judge, then train this to unlock the deeper cost cuts.
- **Sources:**
- FrugalGPT (DistilBERT regression scorer is its quality estimator — confirmed in full text) https://arxiv.org/abs/2305.05176
- RouteLLM (BERT-router surface-feature failure modes) https://arxiv.org/abs/2406.18665

Self-hosted LLM-as-judge scoring (small open-weight judge; on-path only for high-budget requests, otherwise async/offline)

- **Problem it solves:** Subjective qualities — helpfulness, faithfulness, instruction-following — that neither regex nor logprobs nor a DistilBERT head capture reliably.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** GPT-4-class judges reach 85% agreement with human preferences on MT-Bench under the S2 non-tie setup, above the 81% human-human agreement (paper Section 4.2 verbatim; the abstract's headline claim is the more conservative 'over 80%'); weaker/smaller judges score materially lower with known position/verbosity/self-

enhancement biases — reasoned estimate 65..80% agreement for the 7-14B open-weight judges this project can self-host, improvable with strict rubrics and pairwise (not absolute) formats.

- **Latency impact:** +300ms..3s per judged answer (a full generation pass) — the most expensive scorer class; on the critical path it can eat more of the time budget than the answer itself, so gate it behind remaining-budget checks.
- **Cost impact:** +1 small-model inference per judged answer, roughly +10..50% of request compute when judging with a 7-8B model; on owned H100s this is GPU capacity, not API dollars, so it is affordable offline even for a near-zero-budget team.
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — Wrong tool for the per-request critical path (latency + weaker open-weight judges), right tool used asynchronously: labeling logged traffic to train the DistilBERT scorer, calibrating thresholds, and promptfoo rubric assertions — all off the user's clock and on owned GPUs.
- **Sources:**
- Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (Zheng et al., NeurIPS 2023) https://arxiv.org/abs/2306.05685

Golden-set threshold calibration + deadline-aware escalation guard (budget-conditioned thresholds, optional hedged escalation)

- **Problem it solves:** An uncalibrated escalation threshold silently destroys the cascade (escalate-everything = no savings; wave-through = quality collapse), and naive escalation can blow a request's time budget because the hard tail pays double latency.
- **Pipeline stage:** offline
- **Quality impact:** Neutral in expectation but variance-controlling: sweeping the threshold on the 200-500-request promptfoo golden set traces the actual cost-quality Pareto curve and pins regression below a chosen bound (the project's own <5% Phase 2 gate) instead of leaving it accidental; FrugalGPT itself learns thresholds this way on validation data, and vCache ('Verified Semantic Prompt Caching') shows the same error-bounded-calibration pattern works online.
- **Latency impact:** The guard rule — escalate only if remaining time budget >= (elapsed + strong-tier p95 latency), else return best-answer-so-far — converts the escalated tail's +40..110% latency penalty into zero knowing budget violations, at the price of serving the cheap answer on deadline-critical hard requests. Optional hedged escalation (fire the strong tier in parallel once the scorer enters a grey zone): reasoned extrapolation from Dean & Barroso's hedged requests — BigTable 1,000-key read across 100 servers, 99.9th-percentile (not p99) latency cut from 1,800ms to 74ms for ~2% extra requests when hedging after a 10ms delay; that is a distributed-storage read workload, so treat it as an analogy, not a transferable magnitude — suggesting perhaps +10..30% residual escalated-tail penalty for +5..15% extra compute on grey-zone traffic (unvalidated for LLM inference).
- **Cost impact:** Calibration itself ~0 (offline, self-hosted eval runs); the guard prevents cost-budget overruns by construction (skip escalation when remaining cost budget < strong-tier estimate); hedging costs +5..15% compute only where enabled.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Non-optional for THIS project: time and cost budgets are the API contract (must return best-answer-so-far on budget exhaustion), and the promptfoo golden set the calibration needs is already a planned Phase 1 deliverable — this is the piece that makes the cascade budget-safe rather than budget-hostile.
- **Sources:**
- FrugalGPT threshold learning https://arxiv.org/abs/2305.05176
- The Tail at Scale (Dean & Barroso, CACM 2013, hedged requests — 99.9th-percentile figure verified against paper text) https://dl.acm.org/doi/10.1145/2408776.2408794
- vCache: Verified Semantic Prompt Caching (error-bounded calibration pattern) https://arxiv.org/abs/2502.03771
- Project docs: 'AIGateway.docx' requirement 6-4-3 (must not knowingly exceed time/cost budget; return best answer so far — verified in doc, Persian), 'AIGateway (4).docx' §4-5 (200-500-request golden set + <5% Phase 2 gate — verified in doc)

Fact-check verdicts for this area

Claim	Verdict	Note
FrugalGPT matches GPT-4 at up to 98% cost reduction or beats it by ~4% at equal cost; HEADLINES +1.5% accuracy at -80% cost; DistilBERT is the scorer	**confirmed**	Abstract states verbatim 'up to 98% cost reduction' and 'improve the accuracy over GPT-4 by 4% with the same cost' (alternative outcomes, not simultaneous). Full text: HEADLINES case study 'its cost is reduced by 80%, while the accuracy is even 1.5% higher' (Fig. 3c) and 98.3% cost savings at matche
Early-abstention cascade: -2.2% overall test loss, +4.1% abstention rate, -13.0% cost, -5.0% error rate across GSM8K/MedMCQA/MMLU/TriviaQA/TruthfulQA/XSum	**confirmed**	All four numbers and the six-benchmark list match the abstract exactly ('reduces overall test loss by 2.2% on average', '4.1% average increase in the overall abstention rate', '13.0% reduction in cost', '5.0% reduction in error rate'). Caveats added into the technique text: averages across benchmark
Cascade routing: +1..4% over best pure strategy on RouterBench, only +0.3..1.2% on realistic benchmarks; decision overhead 1.6ms (greedy) to ~12ms (full); ICLR	**corrected**	+1..4% on RouterBench confirmed (gap widens with more models, narrows with noise). Realistic-benchmark claim corrected: margins shrink to <=1.2% when quality estimates are poor, but reach up to 14% on SWE-Bench with accurate post-hoc estimates — the 0.3% floor is not in the paper. The 1.6ms/12ms per
GPT-4-class judges reach ~85% agreement with human preferences on MT-Bench, above the 81% human-human agreement	**confirmed**	Exact quote: 'The agreement under setup S2 (w/o tie) between GPT-4 and humans reaches 85%, which is even higher than the agreement among humans (81%).' Setup-specific (non-tie pairs); the abstract's headline is the more conservative 'over 80% agreement, the same level of agreement between humans.'
Dean & Barroso hedged requests: p99 cut from 1800ms to 74ms for +2% load	**corrected**	Numbers real but percentile misattributed: 'sending a hedging request after a 10ms delay reduces the 99.9th-percentile latency for retrieving all 1,000 values from 1,800ms to 74ms while sending just 2% more requests' — it is p99.9, not p99, on a BigTable 1,000-key read across 100 servers. Also a dis
Confidence-based deferral matches trained deferral rules 'remarkably well' in-distribution; fails under specialist models, label noise, distribution shift	**confirmed**	Abstract states 'such confidence-based deferral often works remarkably well in practice' and lists exactly the three failure settings claimed (specialist downstream models, label noise, train/test distribution shift). The 'within ~1-2% of oracle' numeric reading is NOT a paper-stated number — it was
Project-doc attributions: AIGateway__4_.md §3 (FrugalGPT backbone, 98% best-case flag, 50%+ target, 200-500 golden set, <5% gate, 'never refine without a verifi	**corrected**	All cited content verified verbatim in the local docs — including 'Target a FrugalGPT-style 50%+ cost cut', gate '< 5% quality regression on the golden set', 'golden set of ~200-500 representative requests', 'Never refine without a verifier', and requirement 6-4-3 (in Persian): the gateway must not

Caching: exact, prefix/KV, and semantic

vLLM automatic prefix caching (KV-cache reuse)

- **Problem it solves:** Recomputing the KV cache for shared prompt prefixes (system prompts, few-shot templates, multi-turn history) wastes GPU prefill time on every request.
- **Pipeline stage:** execution
- **Quality impact:** neutral — KV blocks are reused bit-exactly, output distribution is unchanged; the only failure mode is a serving-framework bug, not a caching-policy error
- **Latency impact:** TTFT -86% measured on a repeated ~10K-token prompt (4.3s -> 0.6s, Qwen3-32B, single vLLM instance) — a best-case full-prefix hit on an identically resent prompt; the -60..-80% band for chat traffic with 2K+ token shared system prompts is a proportional extrapolation (TTFT savings scale with the cached prefill fraction), not a figure from the cited sources; TPOT is not hurt — SqueezeBits measured TPOT improving ~9.8% alongside the throughput gain; near-zero overhead on miss-only traffic (block-hash lookup)
- **Cost impact:** GPU-seconds per request drop roughly in proportion to the cached prefill fraction; the measured +13.3% throughput (SqueezeBits) is a conservative datapoint — Llama-3.1-8B on 1x A100 with a shared prefix of only 25% of input length, not shared-system-prompt chat on H100 — so longer shared prefixes yield more (TensorRT-LLM gained ~34.7% on the same workload); zero incremental infra cost — it is a config flag (default-on in vLLM v1)
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Free on the exact stack they run (vLLM/H100/KubeAI); zero quality risk; their own engineering brief already mandates turning it on in Phase 1 — the gateway's job is only to maximize prefix sharing

(stable system-prompt templates, PII-redaction placed so it does not break prefix stability).

- **Sources:**
- vLLM docs, Automatic Prefix Caching — https://docs.vllm.ai/en/latest/design/automatic_prefix_caching.html
- SqueezeBits, vLLM vs TensorRT-LLM #12: Automatic Prefix Caching (Dynamic-Sonnet benchmark, shared prefix = 25% of input, Llama-3.1-8B on 1x A100: vLLM +13.3% throughput, TPOT ~+9.8% faster) — https://blog.squeezebits.com/vllm-vs-tensorrtllm-12-automatic-prefix-caching-38189
- llm-d, KV-Cache Wins You Can See (repeated ~10K-token prompt to Qwen3-32B: TTFT 4.3s -> 0.6s) — https://llm-d.ai/blog/kvcache-wins-you-can-see

Prefix-cache-aware routing / KV-aware scheduling across vLLM replicas

- **Problem it solves:** Once a model runs on more than one vLLM replica, random load balancing scatters same-prefix requests across pods so each pod's prefix cache misses, throwing away most of technique 1's benefit.
- **Pipeline stage:** routing-decision
- **Quality impact:** neutral — same models, same outputs; only placement changes
- **Latency impact:** TTFT P90 0.542s with precise KV-aware scheduling vs 92.6s random / 94.9s load-aware / 31.1s approximate (150 tenants x 6K-token contexts, 5 users each, 8 vLLM pods on 16xH100, Qwen3-32B, KV demand 73% of cluster capacity, load ramped to 60 QPS) — note this is a deliberately cache-saturating synthetic stress benchmark, so the ~170x tail gap is best-case; gains shrink toward zero at 1 replica, non-repetitive traffic, or light load
- **Cost impact:** throughput 8,730 vs 6,944 output tok/s (+25%) over approximate scheduling and 1.97x over random/load-aware in the same benchmark; the implied -20..-50% serving cost per token is derived from that cache-heavy stress workload and is an upper bound for production traffic; a cheap consistent-hash/session-affinity approximation in the gateway captures a large share of this for near-zero engineering cost
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — Payoff only materializes once they run >1 replica per model, which a near-zero-budget H100 fleet may not do at first; start with trivial session/tenant consistent-hash affinity in the gateway (ms-scale, stateless-compatible) and adopt llm-d-style precise scheduling only when replica counts grow.
- **Sources:**
- llm-d, KV-Cache Wins You Can See: From Prefix Caching in vLLM to Distributed Scheduling — https://llm-d.ai/blog/kvcache-wins-you-can-see
- Prefix caching in vLLM under multi-tenant agent traffic — https://dev.to/marcuswwchen/prefix-caching-in-vllm-under-multi-tenant-agent-traffic-5e2j

Exact-match response caching (normalized request hashing, Redis tier-0)

- **Problem it solves:** Paying to regenerate byte-identical answers for literally identical requests — client retries, dry-run replays, eval reruns, and templated API-generated traffic.
- **Pipeline stage:** serving-layer
- **Quality impact:** neutral on hits (identical response); only risk is staleness, which TTL policy owns — this makes it the only cache tier with zero false-positive risk
- **Latency impact:** -95..-99% on hits (1-5ms Redis lookup vs 1-10s generation); <1ms added per miss, comfortably inside the ms-scale router-overhead budget
- **Cost impact:** -100% marginal cost per hit; overall savings = hit rate, which is ~0-5% on freeform human chat (paraphrasing defeats exact keys — this is why semantic caching exists) but ~10-30% on retry-heavy, templated, or machine-generated traffic; must normalize keys (whitespace, sampling params, tenant) to get even that
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Trivial to build on the Redis backend req 3.1.1 already mandates, zero quality risk, and the correct tier-0 in front of the semantic cache; also the only cache safe to enable for tenants who opt out of fuzzy matching (req 6.6.4).
- **Sources:**

- MeanCache: User-Centric Semantic Caching for LLM Web Services (motivates: exact matching yields near-zero hits on paraphrased NL traffic) — https://arxiv.org/abs/2403.02694
- GPTCache: An Open-Source Semantic Cache for LLM Applications (same motivation) — https://openreview.net/pdf?id=ivwM8NwM4Z

Semantic response caching (embedding + vector store, GPTCache/MeanCache-style)

- **Problem it solves:** Exact caches miss paraphrases even though a large fraction of real user queries restate earlier intents — MeanCache measured ~31% of user queries as semantically similar to previous ones (self-reported study of 20 ChatGPT power users, ~27K queries — a small-cohort estimate, not production telemetry).
- **Pipeline stage:** serving-layer
- **Quality impact:** -0..-15% correctness on cache-served traffic under naive static thresholds (in MeanCache's eval, GPTCache returned 233 false hits on 700 unique queries; MeanCache cut that to 89, with ~17% higher F-score and 20% higher precision); >97% hit precision is achievable on repetitive categorized traffic with tuned setups — quality risk is entirely a threshold-policy problem (see vCache)
- **Latency impact:** hits served in ~10-120ms vs seconds of generation (-90..-99% per hit); adds ~5-20ms embedding + ANN lookup to EVERY miss — with the ms-scale overhead requirement this forces a co-located small embedding model (CPU MiniLM-class or a slice of GPU), not a remote call
- **Cost impact:** -20..-70% of inference spend BY TRAFFIC TYPE — the widely quoted 30-70% is vendor best-case: the 61.6-68.8% API-call reduction (GPT Semantic Cache) was measured on that paper's own categorized, repetition-heavy query workloads, not general chat; ~31% is a realistic ceiling on general user chat, and even that anchor comes from MeanCache's 20-user self-reported study; low single digits on one-off code/agent traffic. Their own Phase-2 gate (>30% hit rate) is at the optimistic edge and should be re-baselined on logged Iranian-market traffic
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — An explicit project requirement (req 5.2 semantic caching, 6.6.1) and the biggest serving-layer cost lever on repetitive consumer traffic; but it must ship with verified thresholds (vCache), per-tenant scoping, and cache-aware promptfoo evals — a bare GPTCache port would violate the 'never silently regress quality' principle.
- **Sources:**
- MeanCache: User-Centric Semantic Caching for LLM Web Services (31% repeat-query figure from a 20-user/27K-query study; 233-vs-89 false-hit comparison on 700 unique queries) — https://arxiv.org/abs/2403.02694
- GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching (61.6-68.8% API-call reduction, >97% positive-hit precision, Redis) — https://arxiv.org/abs/2411.05276
- GPTCache (OSS reference implementation) — https://github.com/zilliztech/gptcache

vCache verified dynamic per-entry thresholds

- **Problem it solves:** A single global static similarity threshold is provably wrong across domains — the same 0.92 cosine score means 'same intent' for small talk and 'different answer' for math/code — so static caches either forfeit hits or serve wrong answers with no error control.
- **Pipeline stage:** serving-layer
- **Quality impact:** converts unbounded false-positive risk into a user-defined error bound (operator sets max tolerable error rate, e.g. <=1%); up to 26x lower observed error than static-threshold baselines at comparable hit rates (best-dataset maximum from the paper's benchmarks); the error-bound knob maps directly onto the gateway's optional per-request quality target
- **Latency impact:** hit-path latency identical to plain semantic cache; per-entry online threshold learning adds sub-ms bookkeeping; occasional exploration requests still go to the model (that traffic is what the cost budget already covers)
- **Cost impact:** up to 12.5x higher cache hit rate than static-threshold and fine-tuned-embedding baselines at the same error budget (again a best-dataset maximum; typical gains are lower) — i.e. it multiplies technique 4's savings rather than adding its own; no training data or fine-tuning required (matters at near-zero budget)
- **Evidence label:** `evidence-based`

- **Fit for this project: high** — Directly answers this project's central fear (silent quality regression from caching) with a formal knob, is open-source with released benchmarks, needs no labeled data, and its per-entry online learning is stateless-gateway-compatible (state lives in the shared vector store) — this should be the default semantic-cache policy, not an upgrade.
- **Sources:**
- vCache: Verified Semantic Prompt Caching (ICLR 2026; up to 12.5x hit rate, 26x lower error, user-defined error bound — all verified against the arXiv abstract) — https://arxiv.org/abs/2502.03771
- vCache open-source implementation + benchmarks — https://github.com/vcache-project/vCache

Async LLM-judged grey-zone verification (Krites-style tiered static/dynamic cache)

- **Problem it solves:** Requests that land just below the similarity threshold are near-certain paraphrases of curated cached answers but get discarded, and raising the threshold to capture them is exactly the false-positive trap.
- **Pipeline stage:** offline
- **Quality impact:** neutral on the critical path by construction (serving decisions are unchanged; only off-path promotions occur); the reported 99% judge-human agreement is 99/100 pairs on a single 100-pair human-audited sample using Claude Opus 4.5 as a sanity-check judge — the paper's headline results use an oracle judge derived from benchmark equivalence classes rather than an actual LLM, so end-to-end gains assume near-perfect judging; agreement with a self-hosted Qwen-class judge is unverified and will be lower
- **Latency impact:** zero added critical-path latency (verification is async, deduplicated, rate-limited); each approved promotion converts a future near-miss into a ms-scale cache hit
- **Cost impact:** up to 3.9x more requests served from the curated static tier — the 3.9x is the search-style-workload maximum (+290%); conversational traffic gained ~2.4x (+137%) (trace-driven simulations); judge tokens are extra spend but off-path and schedulable onto off-peak self-hosted H100 capacity, so marginal cost is near zero for this team
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — Architecturally cheap for them (idle-GPU judge, background workers, K8s-native queue) and well-suited to FAQ-heavy consumer traffic, but it presupposes a curated static answer tier mined from logs plus real traffic volume, and its published gains rest on oracle-grade judging — a Phase 3+ item, not MVP; validate self-hosted-judge agreement on the promptfoo golden set first.
- **Sources:**
- Krites: Asynchronous Verified Semantic Caching for Tiered LLM Architectures (Apple; arXiv preprint — no peer-review venue listed on arXiv or Apple's research page; a claimed EuroMLSys 2026 acceptance could not be verified) — https://arxiv.org/abs/2602.13165
- Apple ML Research page — https://machinelearning.apple.com/research/semantic-caching

Cache poisoning defense + per-tenant cache isolation

- **Problem it solves:** Semantic cache keys are fuzzy hashes, so an attacker can craft colliding prompts that plant malicious responses served to OTHER users — CacheAttack achieved an 86% average response-hijack hit rate black-box against a GPTCache-style semantic cache, transferred across embedding models at 78.6-86.9% hit rates (white-box >92%), and hit 80.6% on AWS Bedrock and 88.3% on Azure API Management production semantic caches.
- **Pipeline stage:** serving-layer
- **Quality impact:** prevents catastrophic tail failures (attacker-controlled or cross-tenant-leaked answers) rather than lifting average quality; the trade: per-tenant namespacing cuts the achievable hit rate versus a global shared cache (tenant-local repetition < global repetition; estimated -20..-50% relative hit-rate loss, traffic-dependent — extrapolated from MeanCache's per-user-vs-shared framing, not directly measured)
- **Latency impact:** negligible — tenant-ID key prefixing and write-path checks are microseconds; verification-gated writes (only cache responses produced by an authenticated generation, post-PII-redaction) add no serving latency
- **Cost impact:** the hit-rate loss from isolation is the real cost (some % of cache savings forfeited); mitigations: share cache only within a tenant or within a vetted public/anonymous tier, never across paying tenants; near-zero infra cost
- **Evidence label:** `evidence-based`

- **Fit for this project: high** — Non-negotiable for Arvan Cloud's multi-tenant public deployment — reqs 6.6.4 (per-tenant cache config/disable) and 7.5.3 (tenant isolation) already mandate it, and the verified 80-88% success rates against real AWS/Azure production caches make a shared unverified cache an incident waiting to happen; note the attack evidence label covers the attack success rate, while mitigation-effectiveness numbers are reasoned estimates.
- **Sources:**
- From Similarity to Vulnerability: Key Collision Attack on LLM Semantic Caching (CacheAttack: 86% avg hijack hit rate on GPTCache; 80.6% AWS Bedrock; 88.3% Azure API Management; cross-embedding transfer 78.6-86.9%) — https://arxiv.org/abs/2601.23088
- Cache Me, Catch You: Cache-Related Security Threats in LLM Serving Frameworks (NDSS 2026) — https://www.ndss-symposium.org/wp-content/uploads/2026-f2812-paper.pdf

Cache governance: TTL/invalidation policy + cache-hit cost accounting integration

- **Problem it solves:** Without freshness control cached answers silently go stale (a quality bug no threshold catches), and without hits flowing into per-request cost accounting the budget planner and the mandated Cost-Savings/Win-Rate metrics are simply wrong.
- **Pipeline stage:** serving-layer
- **Quality impact:** neutral-to-positive: prevents stale-answer regressions on time-sensitive queries (freshness checking is part of the Krites judge rubric precisely because staleness is the dominant residual failure of verified caches); no direct uplift on fresh traffic
- **Latency impact:** negligible — TTL check and accounting write are microseconds on the hit path
- **Cost impact:** accounting itself is free but load-bearing: a cache hit must be recorded as near-zero marginal cost against the request's cost budget (req 6.6.2) and surfaced as 'served from cache' in the XAI explanation (req 4.1.4) and per-request logs (req 4.1.5) — otherwise reported Cost Savings are fiction; TTL tuning trades savings vs staleness (too short forfeits hits, too long serves stale), and per-tenant/per-policy TTL is required by reqs 3.1.3 and 6.6.1
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Cheap, explicitly mandated by the requirements doc, and the piece that makes every other caching technique's savings measurable and honest — build the accounting hook in Phase 1 alongside the eval harness, before any semantic cache exists, so cache-aware promptfoo runs and dry-run mode can exercise it from day one.
- **Sources:**
- Project requirements doc AIGateway.md reqs 3.1.3, 6.6.1, 6.6.2, 6.6.4, 4.1.4-4.1.5 (design mandate, not external benchmark)
- Krites (freshness as explicit judge criterion) — https://arxiv.org/abs/2602.13165

Fact-check verdicts for this area

Claim	Verdict	Note
vLLM APC: TTFT -60..-80% on 2K+ token shared system prompts; up to -86% (4.3s->0.6s) on repeated ~10K-token prefixes; TPOT 'roughly unchanged (+~10%)'; ~+13% th	**corrected**	The 4.3s->0.6s measurement is confirmed verbatim on the llm-d blog ('a ~10,000 token prompt to a Qwen/Qwen3-32B instance a second time, time-to-first-token drops from 4.3 seconds to just 0.6 seconds') — but it is a best-case full-prefix rehit of an identical prompt, and the model is Qwen3-32B, not Q
KV-aware scheduling: TTFT P90 0.54s precise vs 92.6s random / 31.1s approximate; throughput +25% vs approximate and ~2x vs random/load-aware (8 vLLM pods, 16xH1	**confirmed**	Exact figures on the llm-d blog: P90 TTFT 0.542s / 31.083s / 92.551s (load-scheduling 94.865s); throughput 8,730 vs 6,944.4 output tok/s (+25.7%) vs 4,428.7 tok/s for both cache-blind schedulers (1.97x). Setup matches (8 pods x 2 H100, Qwen3-32B, 150 tenants, 6K-token contexts, 5 users each, KV dema
MeanCache: ~31% of user queries are semantically similar to previous ones; GPTCache 233 false hits vs MeanCache 89 on 700 unique queries; ~17% higher F-score	**confirmed**	All numbers verbatim in the paper: 'Among the 700 unique queries, MeanCache produces only 89 false hits, while GPTCache generates 233 false hits' (test set: 1000 queries, 300 repeated); 'about 31% of user queries were similar to previous ones'; 'approximately 17% higher F-score' (plus 20% higher pre

Claim	Verdict	Note
GPT Semantic Cache: 61.6-68.8% API-call reduction with >97% positive hit rates (Redis)	**confirmed**	Abstract states cache hit rates 'ranging from 61.6% to 68.8%' across query categories and 'positive hit rates exceeding 97%', with embeddings cached in Redis. Measured on the paper's own categorized, repetition-heavy query workloads, not general chat — the finding's best-case framing is retained (it
vCache: up to 12.5x higher cache hit rate and 26x lower error at a user-defined error bound; ICLR 2026	**confirmed**	Abstract confirms 'up to 12.5× higher cache hit' and '26× lower error rates' with user-defined error rate guarantees via per-prompt online threshold learning; ICLR 2026 acceptance confirmed. Both figures carry 'up to' — best-dataset maxima from the paper's benchmarks, which the finding already signa
CacheAttack: 86% black-box response-hijack hit rate, transferable across embedding models, worked against AWS Bedrock and Azure API Management caches	**confirmed**	Paper reports CacheAttack-1 average hit rate over 0.86 (ISR over 0.82) against a GPTCache-based semantic cache, and — contrary to my initial suspicion of overreach — DID run experiments on production services: 80.6% hit rate on AWS Bedrock and 88.3% on Azure API Management (Table 2). Cross-embedding
Krites (Apple, EuroMLSys 2026): up to 3.9x more requests served from the curated static tier; 99% judge-human agreement using a frontier judge	**corrected**	'Up to 3.9 times' confirmed in the abstract, but Table 1 shows it is the search-workload maximum (+290.3%); conversational traffic gained ~2.4x (+136.5%). The 99% figure is 99/100 agreement on a single 100-pair human-audited sample where Claude Opus 4.5 served only as a sanity-check judge — the pape

Prompt compression & context management

> **Area verdict:** VERDICT for this project: NO neural compression in the MVP. Initial traffic is mostly short chat prompts, far below the >5k-token threshold where neural compressors pay off; below it the compressor step (dominated by non-inference algorithmic overhead) adds latency instead of removing it, competes for the same H100 capacity that serves models, breaks prefix/semantic caching, and opens the CompressionAttack surface. MVP gets the free non-neural levers: offline template/few-shot slimming (promptfoo-verified) and prefix-cache-aware context canonicalization. Add CPU-only non-neural condensation on the ROUTER path when long prompts appear (protects the ms-scale router-overhead requirement). Gate LLMLingua-2 / LongLLMLingua behind Phase-3 prompt-length telemetry (only if a real >5k-token / RAG segment emerges), treat code tasks as no-compress by default (even ~1.4x is not proven safe on code), and ship post-compression integrity verification in the same release as the first neural compressor. Abstractive trained compressors are out of scope for a 6-person, 8-month, near-zero-budget team. [Fact-check pass: all primary sources located; headline numbers verified against papers; corrections applied to the code-degradation framing, the router 98x attribution (mostly GPU FlashAttention, not compression), and best-case benchmark caveats on LLMLingua-2/LongLLMLingua/RECOMP figures.]

Static prompt adaptation: few-shot demonstration pruning + system-template slimming (FrugalGPT-style, tuned offline with promptfoo)

- **Problem it solves:** Removes fixed input-token bloat from gateway-injected system prompts and few-shot templates without any runtime compressor model.
- **Pipeline stage:** pre-processing
- **Quality impact:** ~neutral (0..-3%) when every trim is regression-tested against the 200-500-request golden set; FrugalGPT shows reduced demonstration counts can hold accuracy (qualitative — FrugalGPT does not isolate prompt-adaptation savings numerically)
- **Latency impact:** TTFT/prefill -5..-15% on template-heavy short chats (proportional to trimmed tokens); zero added runtime overhead
- **Cost impact:** input-token / prefill GPU-seconds -10..-40% on template-heavy traffic; minimal on raw short user chats with no injected scaffolding
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — The only compression lever that pays on the MVP's mostly-short-chat traffic: zero runtime cost, zero new attack surface, and it reuses the already-planned promptfoo harness. Percentage ranges are extrapolated from typical template-to-user-token ratios (FrugalGPT does not isolate prompt-adaptation savings numerically), hence the label.
- **Sources:**
- FrugalGPT (prompt adaptation component), Chen, Zaharia & Zou 2023 — https://arxiv.org/abs/2305.05176

- Project doc AIGateway (4).docx §3 (prompt adaptation named among FrugalGPT techniques — attribution verified in doc text)

Prefix-cache-aware context canonicalization (static-first ordering; never compress or randomize shared prefixes)

- **Problem it solves:** Orders gateway-assembled context (system prompt, policy injections, PII-redaction scaffolding first; volatile user content last) and keeps it byte-stable so vLLM automatic prefix caching actually hits.
- **Pipeline stage:** pre-processing
- **Quality impact:** neutral
- **Latency impact:** TTFT -30..-80% on requests sharing a long static prefix (hit-rate dependent — cached prefix skips prefill); ~0 on unique prompts; adjacent evidence: SGLang reports 'up to 6.4x higher throughput' vs SOTA serving systems — verified verbatim, but note it is a best-case whole-runtime figure (RadixAttention plus SGLang's DSL/runtime optimizations, across mixed workloads), not a prefix-caching-alone gain and not a TTFT number
- **Cost impact:** prefill GPU compute saved proportional to shared-prefix token share (~-20..-60% prefill FLOPs on template-heavy traffic); zero extra infrastructure
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — MVP, effectively free: Tier-1 strategy already turns on vLLM prefix caching; this is the request-transformation discipline that makes it hit. It is also the argument AGAINST compressing system/template segments — stochastic compression rewrites text per-request and destroys both prefix-cache reuse and semantic-cache normalization.
- **Sources:**
- vLLM Automatic Prefix Caching docs — https://docs.vllm.ai/en/latest/design/prefix_caching.html
- SGLang RadixAttention (adjacent evidence for shared-prefix gains; 6.4x claim verified in abstract) — https://arxiv.org/abs/2312.07104

Non-neural prompt condensation on the router path (TextRank + TF-IDF + U-shaped position weighting + novelty scoring to ~512 tokens, vLLM Semantic Router pattern)

- **Problem it solves:** Keeps router-side intent/complexity classification within the milliseconds-scale overhead budget on long prompts by condensing the CLASSIFIER input (not the LLM input) with zero neural inference in the condensation step itself.
- **Pipeline stage:** routing-decision
- **Quality impact:** neutral on final answer (full prompt still goes to the target model); on routing decisions: domain classification accuracy +8.1pp (53.1%->61.2%) and PII detection +13.9pp (78.5%->92.4%) as denoising side-effect, but jailbreak detection -14.2pp (70.8%->56.6%) — never feed condensed text to safety/jailbreak classifiers (all figures verified verbatim against the paper; single-team self-reported benchmark, not independently replicated)
- **Latency impact:** paper's full stack: router classification 4,918ms -> 50ms at 8K tokens (98x) — but ATTRIBUTION CAVEAT (verified): 38.7x of that comes from moving the classifier to GPU with Flash Attention (co-located, <800MB footprint, AMD GPUs in the paper); condensation itself contributes only 2.0x (127ms -> 62ms) and near-streaming 1.2x. The condensation step is genuinely CPU-only (~19ms at 16K-token input); a pure-CPU deployment of the whole classifier stack forgoes the 38.7x FA gain. No benefit on short prompts, where Arch-Router-1.5B classification is already fast
- **Cost impact:** near-zero for the condensation step (CPU-only); the paper's headline latency additionally assumes the classifier shares an existing GPU (<800MB footprint) rather than zero GPU involvement — still no dedicated GPU provisioning
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Directly protects hard requirement 4.2.1 (router overhead negligible, ms-scale) once long prompts appear, at near-zero budget — Arch-Router-1.5B inference time grows with prompt length and condensation caps the classifier input; adopt when p95 prompt length crosses a few thousand tokens, not MVP day one. Note the 98x headline is mostly the GPU+FlashAttention classifier, so plan for a small co-located GPU slice, not a CPU-only sidecar, if the full speedup is required.
- **Sources:**

- 98x Faster LLM Routing Without a Dedicated GPU (vLLM Semantic Router) — https://arxiv.org/html/2603.12646v1
- Project lit review AIGateway (2).docx (WRP / vLLM Semantic Router section)

Multi-turn conversation-history summarization (recursive summarization with pinned system state + verbatim recent-K turns)

- **Problem it solves:** Stops the quadratic cumulative-token growth of stateful sessions (requirement 1.1.3) so long conversations stay within per-request cost/time budgets instead of hitting context limits or draining the cost budget on history re-prefill.
- **Pipeline stage:** pre-processing
- **Quality impact:** 0..-10% on long-horizon coherence; risk concentrated in the survey's F1/F2/F3 failure modes (wrong preservation decisions, loss of temporal/actionable state, unrecoverable data) — mitigated by pinning system prompt + last K turns verbatim and summarizing only older history
- **Latency impact:** adds one small-model summarization call every N turns; run it async between turns for ~0 user-visible latency; reduces prefill on every subsequent turn; caveat: each re-summarization invalidates the session's vLLM prefix-cache reuse, so schedule at coarse N-turn boundaries
- **Cost impact:** cumulative session token cost -30..-70% on sessions >20 turns (quadratic -> ~linear growth arithmetic, not a benchmarked figure); negligible on 1-5-turn chats
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: medium** — Not MVP (mostly single-shot traffic), but becomes high the moment stateful multi-turn sessions see >10-20 turns — and it is cheap: the summarizer is a small Qwen already in the fleet. No single benchmark quantifies the savings, hence the label; the range is token-growth arithmetic plus the survey's qualitative findings.
- **Sources:**
- Context Compression for LLM Agents: A Survey of Methods, Failure Modes, and Evaluation — https://www.preprints.org/manuscript/202605.2065
- LiteLLM v1.83.14 server-side prompt compression callback (production precedent; release note verified — note it is a generic server-side compression callback, not conversation-summarization-specific) — https://docs.litellm.ai/release_notes/v1.83.14/v1-83-14

Extractive task-agnostic compression at dispatch (LLMLingua-2, XLM-RoBERTa-large token classifier distilled from GPT-4)

- **Problem it solves:** Cuts prompt tokens 2-5x before dispatch to the target model, saving prefill compute and letting oversized inputs fit inside tight time/cost budgets.
- **Pipeline stage:** pre-processing
- **Quality impact:** ~0..-5% task score at 2x-5x compression on task-agnostic benchmarks (MeetingBank, LongBench, ZeroSCROLLS, GSM8K, BBH), sometimes beating uncompressed baselines — authors' own benchmark suite, best-case conditions; HARD EXCEPTION (corrected): on code/structured tasks the in-the-wild study finds 'even slight compression harms model outputs' (edit-similarity drops), with practical ratios capped ~1.4x on LongBench LCC — 1.4x is a practicality ceiling, NOT a proven-safe threshold, so treat code as no-compress by default, gated by task class from the router
- **Latency impact:** paper claims end-to-end 1.6x-2.9x speedup (-38..-66%) at 2x-5x ratios (verified verbatim; authors' best-case datacenter conditions) — independent in-the-wild replication (30,000 queries, 3 GPU classes) finds only up to 18% e2e speedup, and only when prompt length, compression ratio, and hardware are matched, with largest gains beyond ~5k-token prompts; on short prompts NET LATENCY INCREASES because non-inference algorithmic overhead around the model call dominates compressor latency (model inference is only ~5% of it); compressor 3x-6x faster than original LLMLingua (verified); consumer GPU (GTX 1080 vs A100) ~3x slower, Apple M1 Pro ~5x slower (verified)
- **Cost impact:** input-token / prefill cost -50..-80% at 2x-5x on long prompts (arithmetic from ratio; for self-hosted: H100 prefill-seconds freed), minus hosting the ~560M-param compressor; also relevant for metered closed-API fallback calls when those are reachable
- **Evidence label:** `evidence-based`

- **Fit for this project: medium** — Defer to Phase 3, gated on logged prompt-length telemetry showing a real >5k-token segment: MVP short-chat traffic sits below the payoff threshold, the compressor competes for serving GPU capacity, realistic production speedup is closer to the replication's 18% ceiling than the paper's 2.9x, and it requires the task-class gate (code = no-compress) plus the security hardening entry below before production.
- **Sources:**
- LLMLingua-2 — https://arxiv.org/abs/2403.12968
- Prompt Compression in the Wild: Measuring Latency, Rate Adherence, and Quality — https://arxiv.org/abs/2604.02985
- LLMLingua project page — https://llmlingua.com/llmlingua2.html

Question-aware long-context/RAG compression with document reordering (LongLLMLingua)

- **Problem it solves:** Removes retrieval noise and mitigates lost-in-the-middle by contrastive-perplexity token scoring and most-relevant-first/last document reordering, making long RAG prompts simultaneously cheaper AND more accurate.
- **Pipeline stage:** pre-processing
- **Quality impact:** up to +21.4% accuracy on multi-document QA (NaturalQuestions, GPT-3.5-Turbo) at ~4x fewer tokens (verified verbatim — note this is the paper's best-case headline; no lower bound is given, and gains are task-dependent, shrinking outside retrieval-noise-heavy QA) — one of the few levers that can RAISE quality while cutting cost
- **Latency impact:** end-to-end 1.4x-2.6x speedup (-29..-62%) on ~10k-token prompts at 2x-6x ratios (verified verbatim; a previously cited 3.8x figure could not be located in either the paper or the project docs and has been removed); same >5k-token viability caveat as LLMLingua-2 — below that the compressor step dominates
- **Cost impact:** -75% tokens at 4x compression (arithmetic); 94.0% cost reduction reported on the LooGLE long-context benchmark (verified verbatim — best-case benchmark token accounting on an extreme-length benchmark, unlikely to transfer at that magnitude to mixed production traffic)
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — High value IF the gateway serves RAG traffic (the data-layer requirement 3.3.1 vector-DB hooks anticipate it), irrelevant to MVP chat; its perplexity scorer needs only a small self-hosted LM, which the Qwen/Llama fleet already provides — the rare technique whose quality-positive direction aligns with the project's quality-maximization objective.
- **Sources:**
- LongLLMLingua (ACL 2024) — https://arxiv.org/abs/2310.06839
- LlamaIndex LongLLMLingua integration guide — https://www.llamaindex.ai/blog/longllmlingua-bye-bye-to-middle-loss-and-save-on-your-rag-costs-via-prompt-compression-54b559b9ddf7

Compression security hardening: post-compression semantic-integrity verification + SafeContext-style control pinning + PPL anomaly detection

- **Problem it solves:** Prevents CompressionAttack-class exploits where adversarial input steers the compressor into discarding safety instructions or synthesizing fragmented malicious payloads (relinking / confused-deputy), bypassing the gateway's PII/injection filters that run BEFORE compression.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** neutral on benign traffic; the threat it blocks is quantified: COMA transfer attack achieves 0.71 average attack success rate vs 0.21 for the strongest baseline across 3 tasks x 6 compressors (verified verbatim against the arXiv abstract; the ~87% best-case ASR figure is confirmed present in the project lit review, sourced from an earlier/expanded version, not the current abstract); defense efficacy itself is not yet well quantified — SecAlign's <10% injection-success figure is from general prompt-injection defense, not compression-specific
- **Latency impact:** +1..+10ms per request for an embedding-similarity pre/post-compression integrity check; LLM-judge escalation adds a model call — reserve for flagged requests only
- **Cost impact:** near-zero: reuses the co-located embedding model the semantic cache already requires; pinning system/safety prefixes is free configuration

- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Conditionally mandatory: must ship in the SAME release as the first neural compressor (LLMLingua-2/LongLLMLingua), because the project's PII-redaction-before-dispatch pipeline places all filters exactly where these attacks bypass them; zero work needed while the MVP contains no compression. Attack magnitude is evidence-based (verified); the label reflects that defense-side numbers are extrapolated.
- **Sources:**
- CompressionAttack — https://arxiv.org/abs/2510.22963
- Safe to Check, Unsafe to Use: Relinking at the Compression Boundary of LLM Agents — https://arxiv.org/html/2606.21732v1
- SecAlign — https://arxiv.org/abs/2410.05451

Abstractive trained compression for RAG (RECOMP; SCOPE / Cmprsr for rate-controlled variants)

- **Problem it solves:** Achieves extreme compression (down to ~6% of original tokens) by training a model to synthesize multi-document retrievals into one dense summary — including returning an empty string for irrelevant retrievals (selective augmentation) — instead of dropping individual tokens.
- **Pipeline stage:** pre-processing
- **Quality impact:** RECOMP reports 'a compression rate of as low as 6% with minimal loss in performance' on language modeling and open-domain QA (verified verbatim — the paper gives NO numeric loss figure; the ~0..-5% band previously stated is an analyst estimate, and 'as low as 6%' is the best case, with typical rates higher, i.e. roughly 6-17x compression by arithmetic); beyond ~5x ratios abstractive is claimed to beat extractive and off-the-shelf LLM summarizers fail rate adherence badly (Cmprsr benchmark across 25 LLMs — post-cutoff paper, claims not independently verified), so a trained compressor is required, not optional
- **Latency impact:** adds a full generation pass by a ~4B compressor (hundreds of ms to seconds); net latency win only when saved target-model prefill exceeds compressor decode — very long prompts into large/slow targets
- **Cost impact:** token cost -80..-94% at 6-17x on RAG traffic (arithmetic from compression rate, best case at the 6% rate), minus compressor inference (e.g. Cmprsr's SFT+GRPO-trained Qwen3-4B occupying GPU capacity)
- **Evidence label:** `evidence-based`
- **Fit for this project: low** — Wrong effort profile for this team: requires training/fine-tuning and hosting a dedicated compressor per task family — SFT+RL pipelines a 6-person, near-zero-budget team cannot afford inside the 8-month roadmap; reconsider after Phase 4 only if a heavy RAG product line materializes and LongLLMLingua (training-free) proves insufficient.
- **Sources:**
- RECOMP — https://arxiv.org/abs/2310.04408
- Cmprsr — https://arxiv.org/abs/2511.12281
- SCOPE — https://arxiv.org/abs/2508.15813

Fact-check verdicts for this area

Claim	Verdict	Note
LLMLingua-2: 2x-5x compression, end-to-end latency 1.6x-2.9x faster, compressor 3x-6x faster than prior methods, on MeetingBank/LongBench/ZeroSCROLLS/GSM8K/BBH	**confirmed**	Abstract states verbatim: 'compression ratios of 2x-5x', 'accelerating the end-to-end latency by 1.6x-2.9x', 'model is 3x-6x faster than existing prompt compression methods', benchmarks 'MeetingBank, LongBench, ZeroScrolls, GSM8K, and BBH'. These are the authors' best-case datacenter numbers; the in
In-the-wild replication: up to 18% e2e speedup only when length/ratio/hardware match; largest gains beyond ~5k-token prompts; ~95% of compressor latency is post	**confirmed**	Verified: 'LLMLingua achieves up to 18% end-to-end speed-ups, when prompt length, compression ratio, and hardware capacity are well matched'; 'the largest gains beyond ~5k-token prompts'; 'model inference accounts for only about 5% of total latency' (so ~95% is overhead); GTX 1080 vs A100 ~3x total
Code/structured tasks degrade severely (edit similarity collapse) beyond ~1.4x compression, so code is safe below that ratio	**corrected**	Misreads the paper in the optimistic direction. The paper says 'significant drops in edit similarity suggest that even slight compression harms model outputs' on code, and that code tasks force 'low compression ratios (e.g., 1.4x for the LongBench LCC task), which limit both compression benefit and

Claim	Verdict	Note
LongLLMLingua: +5..+21.4% accuracy on NaturalQuestions at ~4x fewer tokens; 1.4x-2.6x e2e speedup on ~10k-token prompts at 2x-6x; 94.0% cost reduction on LooGLE	**corrected**	Paper confirms verbatim: 'boosts performance by up to 21.4% with around 4x fewer tokens in GPT-3.5-Turbo', 'accelerate end-to-end latency by 1.4x-2.6x' 'when compressing prompts of about 10k tokens at ratios of 2x-6x', 'achieves a 94.0% cost reduction in the LooGLE benchmark'. Two fixes: (1) the '+5
vLLM Semantic Router: 4,918ms -> 50ms at 8K tokens (98x), breakdown Flash Attention 38.7x / compression 2.0x / near-streaming 1.2x; ~19ms CPU condensation at 16	**corrected**	Every number verified verbatim, including the jailbreak-detection drop. But the attribution inside the findings was skewed: the 98x total is dominated by moving the classifier to GPU with Flash Attention (38.7x, on AMD GPUs, <800MB co-located footprint) — the CPU condensation itself contributes only
CompressionAttack: COMA transfer attack achieves 0.71 average ASR vs 0.21 for the strongest baseline across 3 tasks x 6 compressors; up to ~87% ASR per the proj	**confirmed**	Abstract states verbatim: 'COMA achieves 0.71 average ASR, versus 0.21 for the strongest baseline' 'across three tasks and six compressors'. The 87% figure is genuinely present in the project lit review ('targeted attacks achieve up to an 87% attack success rate (ASR) across multiple LLMs') and is c

Parallel execution: best-of-N, ensembles, speculative/hedged dispatch

Best-of-N sampling + self-consistency (majority vote)

- **Problem it solves:** A single greedy pass underperforms on hard reasoning tasks; sampling N diverse chains and voting on the final short answer recovers accuracy the model already has.
- **Pipeline stage:** execution
- **Quality impact:** +4..+18 points absolute accuracy on reasoning benchmarks (GSM8K +17.9, AQuA +12.2, SVAMP +11.0, StrategyQA +6.4, ARC-c +3.9 — verified verbatim against the paper's abstract). Best-case caveat: these headline deltas were measured with ~40 sampled paths on 2022-era base models (PaLM-540B/GPT-3-class); modern instruction-tuned models start from much higher single-pass accuracy and typically show smaller absolute deltas — treat the top of the range as ceiling, not expectation. Most of the gain arrives by N=5-10 per the paper's scaling curves. Holds ONLY for tasks with an extractable, votable short answer (math, MCQ, classification, extraction); ~neutral-to-negative on open-ended chat where no vote is possible.
- **Latency impact:** +0..+30% wall-clock when the N samples run as one parallel batch on self-hosted vLLM at low-to-moderate GPU utilization (n>1 shares the prefill); degrades toward +100%+ under saturation because N samples consume N decode slots and grow queueing delay.
- **Cost impact:** Linear in N on decode: N=5 => roughly +300..+400% GPU-time vs one pass (prefill amortized on vLLM). In dollars on self-hosted idle capacity it is cheap; in throughput/opportunity cost it is a real xN.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Cheapest quality dial for a team with self-hosted models and no training data; already Tier-2 in the strategy brief. Decision predicate: task has a votable short answer AND difficulty heuristic flags hard AND cost budget >= N x baseline AND time budget allows one parallel batch — else fall back to single routed pass.
- **Sources:**
- Self-Consistency Improves Chain of Thought Reasoning in Language Models (Wang et al., ICLR 2023) — https://arxiv.org/abs/2203.11171

Verifier-gated best-of-N (pick-best with a checkable signal)

- **Problem it solves:** For non-votable outputs (code, JSON, proofs), extra samples only pay off if a reliable picker exists; a verifier (tests, compiler, schema, reward model) turns N samples into a large accuracy gain.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** With deterministic verifiers: SWE-bench Lite 15.9% -> 56% of issues solved at 250 samples with DeepSeek-Coder-V2-Instruct (Large Language Monkeys, verified) — CAVEAT: 56% is coverage, counting an issue solved if ANY sample passes the benchmark's held-out ground-truth tests; that oracle verifier does not exist in production, where realized gain is bounded by the quality of your own test suite. GSM8K: a 6B model sampling 100 completions ranked by a trained 6B verifier slightly outperforms a finetuned 175B model — 'a boost approximately equivalent to a 30x model size increase' (Cobbe 2021, quote verified; note the picker is a trained verifier, not a free deterministic check). MATH: PRM best-of-N 78.2% vs 72.4% ORM vs 69.6% majority vote (+8.6 pts, Lightman 2023,

verified) — but at best-of-1860 with a GPT-4-scale PRM; the PRM-vs-majority gap shrinks sharply at online N=3-10. The same paper family finds majority voting and reward-model pickers plateau beyond several hundred samples. With a weak LLM judge as picker the gain shrinks to a few points and can invert. Practical online N=3-10 with deterministic checks captures a minority of the benchmark-max gain (estimate, not measured).

- **Latency impact:** +5..+20% over plain parallel best-of-N (one extra cheap scoring pass over candidates); massive-N (100+) is offline/batch-only, not compatible with interactive time budgets.
- **Cost impact:** Linear in N plus verifier cost (deterministic checks ~free; judge model +5..+10%). Online N=3-10 => +200..+900% vs single pass; only justified on the hard, checkable slice of traffic.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Exactly the strategy brief's 'never pay for N>1 without a checkable signal' rule; deterministic verifiers (does code run / JSON validate) are free to build, while training a PRM is out of budget. Decision predicate: automatic verifier exists AND verifier cost << generation cost AND request flagged hard AND cost budget >= N x baseline; N sized to remaining cost budget. Note the headline benchmark numbers all use oracle or trained verifiers — expect materially smaller gains from homegrown deterministic checks.
- **Sources:**
- Training Verifiers to Solve Math Word Problems (Cobbe et al. 2021, GSM8K) — https://arxiv.org/abs/2110.14168
- Let's Verify Step by Step (Lightman et al. 2023, MATH/PRM800K) — https://arxiv.org/abs/2305.20050
- Large Language Monkeys: Scaling Inference Compute with Repeated Sampling (Brown et al. 2024, SWE-bench Lite) — https://arxiv.org/abs/2407.21787

Mixture-of-Agents / LLM-Blender fusion ensemble

- **Problem it solves:** No single open-weight model dominates all prompts; layering several open models as proposers plus an aggregator (or ranking+fusing candidates) can beat the best single model — even closed frontier models — on chat quality.
- **Pipeline stage:** execution
- **Quality impact:** MoA with only open-source models: AlpacaEval 2.0 length-controlled win rate 65.1% vs GPT-4o's 57.5% (+7.6 pts; verified — Table 2a, LC column; plain win rates 59.8% vs 51.3%); also claims SOTA on MT-Bench/FLASK. The headline config is 6 proposers x 3 layers with Qwen1.5-110B-Chat as aggregator. LLM-Blender on MixInstruct: avg GPT-Rank 3.01 vs best single model (Open Assistant) 3.90; top-3 rate 68.59% vs 51.98% (corrected — previously misquoted as 52.9%). Conditions: gains are LLM-judged preference win rates on open-ended instruction traffic, not correctness — this metric rewards style and is judge-sensitive, so treat as best-case for real chat quality; on strictly factual/verifiable tasks fusion can propagate shared errors and verifier-gated BoN dominates.
- **Latency impact:** +100..+200% end-to-end (2-3 sequential rounds: proposer layer(s) then aggregator); irreducible because the aggregator must wait for all proposers.
- **Cost impact:** +300..+600% (MoA-Lite ~6-7 calls; full 3-layer MoA with 6 proposers 10-18 calls). Fusion (LLM-Blender) adds ranker + fuser on top of N base generations.
- **Evidence label:** `evidence-based`
- **Fit for this project: low** — Strategy brief Tier-3 skips consensus ensembles (3x+ cost for a few points); the one redeeming scenario here is that MoA lets a self-hosted Qwen/Llama fleet exceed closed-API chat quality when sanctions cut API access. Decision predicate: explicit max-quality target AND cost budget >= 5-10x AND time budget >= 2-3x single pass AND open-ended task with no cheap verifier — expected to be a rare traffic slice; implement last, if ever.
- **Sources:**
- Mixture-of-Agents Enhances Large Language Model Capabilities (Wang et al. 2024, AlpacaEval 2.0/MT-Bench) — https://arxiv.org/abs/2406.04692
- Together MoA reference implementation — https://github.com/togethercomputer/moa
- LLM-Blender: Ensembling LLMs with Pairwise Ranking and Generative Fusion (Jiang et al., ACL 2023, MixInstruct) — https://arxiv.org/abs/2306.02561

Multi-model debate (rejected)

- **Problem it solves:** Tries to improve factuality/reasoning by having several model instances critique each other's answers over multiple rounds and converge.
- **Pipeline stage:** execution
- **Quality impact:** +5..+10 points absolute on GSM8K/MMLU with 3 agents x 2 rounds in the original paper — BUT the controlled ICML 2024 comparison (Smit et al.) finds multi-agent debate does NOT reliably outperform self-consistency/ensembling at matched compute and is highly hyperparameter-sensitive. Net vs the self-consistency alternative at equal budget: ~0, sometimes negative.
- **Latency impact:** +200..+400% (rounds are inherently sequential; each round waits on all agents).
- **Cost impact:** +500..+800% (agents x rounds; 3x3 = 9 full generations plus growing context per round).
- **Evidence label:** `evidence-based`
- **Fit for this project: low** — Dominated: same budget spent on self-consistency or verifier-gated BoN buys equal or better quality with parallel (not sequential) latency — which is why the strategy brief's Tier-3 explicitly rejects it. Decision predicate: none — the gateway should never select debate; route any generous budget to self-consistency/verifier-BoN instead.
- **Sources:**
- Improving Factuality and Reasoning in Language Models through Multiagent Debate (Du et al., ICML 2024) — https://arxiv.org/abs/2305.14325
- Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs (Smit et al., ICML 2024) — https://proceedings.mlr.press/v235/smit24a.html

Hedged request dispatch (duplicate to replica after p95 delay, cancel loser)

- **Problem it solves:** Endpoint tail latency (GPU queueing spikes, flaky sanction-routed closed APIs) blows the request's time budget even when median latency is fine; a deferred duplicate request caps the tail.
- **Pipeline stage:** serving-layer
- **Quality impact:** Neutral — same model or equivalent replica serves the answer; quality is unchanged by construction.
- **Latency impact:** In the canonical non-LLM benchmark (verified against the paper's full text): 99.9th-percentile latency -96% (1800ms -> 74ms) for a 1,000-key read fanned out across 100 BigTable servers, hedging after a fixed 10ms delay. Best-case caveat: that is a 100-way fan-out read, the regime where hedging pays most; a single LLM endpoint request has no fan-out, so relative tail gains will be smaller. For LLM endpoints expect p99 -30..-70% when endpoint variance is high — extrapolated, not measured on LLM serving; LLM adaptation is TTFT-based hedging (fire hedge if no first token by p95 TTFT).
- **Cost impact:** +2..+5% extra requests. Attribution corrected: the paper measured ~2% extra requests for the fixed-10ms-delay BigTable benchmark, and separately bounds the defer-until-p95-expected-latency policy at approximately 5% additional load. Naive immediate duplication would be +100%, so always defer; cancellation must actively abort the losing vLLM request to reclaim GPU, else cost doubles silently.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Directly enforces the 'never knowingly exceed the time budget' contract against exactly this project's failure modes (unreliable closed APIs, bursty GPU queues); trivial in an async stateless gateway and near-free at +2-5% cost. Decision predicate: remaining time budget < p99 latency of chosen endpoint OR endpoint health-variance high; defer hedge until p95 TTFT elapses; max 1 hedge; only across replicas/endpoints of equal quality tier.
- **Sources:**
- The Tail at Scale (Dean & Barroso, CACM 2013) — https://cacm.acm.org/research/the-tail-at-scale/ (full text verified via https://www.barroso.org/publications/TheTailAtScale.pdf)
- No published LLM-gateway hedging study found as of 2026-07 (searched); LLM transfer is extrapolation

Speculative cheap+strong dispatch (parallel cascade across model tiers)

- **Problem it solves:** A sequential FrugalGPT cascade pays the cheap model's full latency before escalating; under a tight time budget the gateway cannot afford cheap-then-strong in series, so it fires both in parallel and cancels the strong model if the cheap answer passes the scorer.
- **Pipeline stage:** execution
- **Quality impact:** Same as the sequential cascade with the same acceptance scorer: retains ~90-100% of strong-model quality. FrugalGPT's abstract claim verified: 'match the performance of the best individual LLM (e.g. GPT-4) with up to 98% cost reduction or improve the accuracy over GPT-4 by 4% with the same cost' — note the 'up to': it is a best-case figure on a handful of narrow benchmark tasks with a per-task trained acceptance scorer, and does not transfer to heterogeneous production traffic without comparable scorer quality. Equivalently +5..+15 points vs cheap-only on escalated traffic. Token-level speculative cascades (Google, ICLR 2025) confirm speculation+deferral beats plain cascades on cost-quality, but at the inference-engine layer, not gateway layer.
- **Latency impact:** Removes the escalation penalty on escalated requests: worst case drops from (cheap latency + strong latency) to max(strong latency), i.e. -30..-50% on the escalated slice; accepted-cheap requests unchanged.
- **Cost impact:** Vs sequential cascade: +10..+40% on hedged requests (strong model's prefill + partial decode wasted whenever the cheap answer is accepted; early cancel keeps it low). Vs always-strong: still -40..-80%. On self-hosted idle GPU the dollar cost of waste is small; the throughput cost is real under load.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — The natural time-budget-aware upgrade of the project's FrugalGPT backbone — it is what makes the cascade usable when the time budget is too tight for sequential escalation. Decision predicate: router predicts cheap probably suffices AND time budget < (cheap + strong) sequential latency AND cost budget tolerates possible double execution AND a fast scorer exists to accept/reject the cheap answer; otherwise run the cascade sequentially.
- **Sources:**
- FrugalGPT (Chen, Zaharia, Zou 2023) — https://arxiv.org/abs/2305.05176
- Faster Cascades via Speculative Decoding (Narasimhan et al., ICLR 2025) — https://arxiv.org/abs/2405.19261
- The Tail at Scale (hedging pattern) — https://cacm.acm.org/research/the-tail-at-scale/; gateway-level combination is extrapolated, no direct benchmark

Racing semantic-cache lookup against generation

- **Problem it solves:** A serial semantic-cache lookup (embed + vector search, ~5-50ms) taxes every cache miss and violates the ms-scale router-overhead requirement; racing lookup and generation concurrently makes the miss penalty zero.
- **Pipeline stage:** serving-layer
- **Quality impact:** Neutral, conditional on a verified cache policy (vCache-style dynamic thresholds); the quality risk of false cache hits belongs to the caching decision itself, and racing neither adds nor removes it.
- **Latency impact:** Hit traffic: -90..-99% (tens of ms vs seconds of generation). Miss traffic: 0ms added (vs +5-50ms with a serial blocking lookup). Net effect scales with hit rate (project docs cite 30-70% on repetitive traffic — vendor/self-reported figures, not independently benchmarked).
- **Cost impact:** Savings = hit rate x generation cost avoided (30-70% on repetitive traffic per GPTCache-class reports and the project's own strategy brief — best-case, repetitive-workload figures); racing overhead is the wasted prefill on hits that resolve after generation started — ~1-5% of hit-traffic generation cost if the gateway cancels within the first tokens.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Semantic caching is an explicit project requirement and the ms-overhead constraint forbids a blocking lookup, so racing (or full async lookup) is effectively the mandated design in the async stateless core. Decision predicate: always race when the request is cacheable (no PII flags, deterministic-enough policy, cache enabled for tenant); skip the race and go serial only if measured lookup p99 < ~5ms.
- **Sources:**
- vCache: Verified Semantic Prompt Caching (2025) — https://arxiv.org/abs/2502.03771

- GPTCache (open-source semantic cache, reported hit rates) — https://github.com/zilliztech/GPTCache
- Race pattern extrapolated from The Tail at Scale hedging — https://cacm.acm.org/research/the-tail-at-scale/; no paper measures the race variant for LLM caches

Iterative refine loop (draft -> check -> fix) with verifier + max-turns cap

- **Problem it solves:** First-pass code/structured output often contains mechanical errors a cheap external check (tests, compiler, JSON schema) can catch, and one or two feedback turns fix most of them.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** Attribution corrected: Self-Refine's ~+20 points absolute average across 7 tasks (GPT-3.5/ChatGPT/GPT-4; abstract verified) is achieved with SELF-feedback — the same model as generator, feedback provider, and refiner — mostly on open-ended, preference-scored tasks, and its math-reasoning gain was only ~+0.2; it does NOT evidence external-feedback loops. The number that does: Reflexion with external test signals reaches 91% vs 80% pass@1 on HumanEval (~+11 points, verified). Realistic expectation for this technique with reliable deterministic checks (tests/compiler/schema): ~+10 points on the code/structured slice. Hard condition intact: WITHOUT an external signal, intrinsic self-correction on reasoning is ~0 to negative (Huang et al. 2023 show degradation) — never refine on vibes.
- **Latency impact:** +100..+300% (2-4 inherently sequential turns; each turn = verify + regenerate). Must be bounded by remaining time budget with best-so-far returned on expiry.
- **Cost impact:** +100..+300% (roughly +1x draft cost per extra turn plus near-free deterministic verification; cap at 2-3 turns — returns diminish sharply after turn 2).
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Already Tier-2/Phase-3 in the strategy brief for the code/structured-output slice, and its budget-capped loop maps one-to-one onto the return-best-answer-so-far contract. Decision predicate: automatic verifier exists AND draft FAILED verification AND turns < max-turns cap (2-3) AND remaining time+cost budget covers >= 1 more turn; on budget expiry return best-verified-so-far draft.
- **Sources:**
- Self-Refine: Iterative Refinement with Self-Feedback (Madaan et al., NeurIPS 2023) — https://arxiv.org/abs/2303.17651
- Large Language Models Cannot Self-Correct Reasoning Yet (Huang et al. 2023) — https://arxiv.org/abs/2310.01798
- Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., NeurIPS 2023) — https://arxiv.org/abs/2303.11366

Fact-check verdicts for this area

Claim	Verdict	Note
Self-consistency gains: GSM8K +17.9, AQuA +12.2, SVAMP +11.0, StrategyQA +6.4, ARC-c +3.9 points absolute	**confirmed**	Abstract matches verbatim: 'GSM8K (+17.9%), SVAMP (+11.0%), AQuA (+12.2%), StrategyQA (+6.4%) and ARC-challenge (+3.9%)'. Caveat added to impact text: these headline deltas were measured with ~40 sampled paths on 2022-era base models (PaLM-540B/GPT-3-class); modern instruction-tuned models start fro
Large Language Monkeys: SWE-bench Lite 15.9% -> 56% solved at 250 samples 'picked by unit tests'	**confirmed**	Numbers confirmed: DeepSeek-Coder-V2-Instruct, 15.9% one-sample -> 56% at 250 samples, vs 43% single-sample SOTA. Correction folded into impact text: 56% is COVERAGE — an issue counts if ANY sample passes the benchmark's held-out ground-truth test suite, which is an oracle verifier unavailable in pr
Lightman 2023: MATH best-of-N via PRM 78.2% vs 69.6% majority vote (+8.6 pts)	**confirmed**	Exact numbers confirmed: PRM 78.2%, ORM 72.4%, majority voting 69.6%. Material omission fixed: these are best-of-1860 results (N=1860 samples per problem) with a GPT-4-scale PRM trained on PRM800K, and the PRM-vs-majority gap widens with N — at online N=3-10 the gap is far smaller. Condition now sta
MoA (open-source only): AlpacaEval 2.0 LC win rate 65.1% vs GPT-4o 57.5%	**confirmed**	Confirmed, and the 'LC win rate' attribution is correct: Table 2(a) lists MoA 65.1±0.6% LC win (59.8% plain win) vs GPT-4 Omni (05/13) 57.5% LC (51.3% plain). Headline config is 6 proposers x 3 layers with Qwen1.5-110B-Chat aggregator, consistent with the stated cost multiple. Sub-claim corrected: L

Claim	Verdict	Note
Tail at Scale hedging: p999 1800ms -> 74ms after 10ms hedge on a 100-server BigTable read; deferred-to-p95 hedging measured ~2% extra requests	**corrected**	The 1,000-key/100-server BigTable numbers are confirmed verbatim (99.9th pct 1800ms -> 74ms, 'just 2% more requests'). Attribution corrected: the ~2% figure belongs to that fixed-10ms-delay benchmark, while the defer-until-p95-expected-latency policy is what the paper bounds at 'approximately 5%' ad
Iterative refine loop: '~+20 points absolute average across Self-Refine's 7 tasks' attributed to reliable external/deterministic feedback	**corrected**	Misattribution. The ~20% absolute average is real (abstract: 'improving by ~20% absolute on average in task performance', 7 tasks, GPT-3.5/ChatGPT/GPT-4) but it is achieved with SELF-feedback — the same model as generator, feedback provider, and refiner, no external signal — mostly on open-ended pre

Cost-aware scheduling, budget enforcement, and serving-layer levers

Per-request budget ledger with deadline/cost propagation and a graceful-degradation ladder (best-so-far semantics)

- **Problem it solves:** The gateway must never knowingly exceed a request's time or cost budget (reqs 6.4.3, 7.1.1, 7.3.1) and must return the best answer obtained so far when a budget runs out mid-flight, which requires every downstream call to carry a timeout equal to the remaining time budget (req 6.5.3) and every retry/escalation/sample to debit a shared ledger.
- **Pipeline stage:** execution
- **Quality impact:** Neutral on requests that finish within budget (the large majority if length prediction works); on the est. 3-10% of requests whose budget would otherwise be blown, converts a total failure (quality = 0, error returned) into a best-so-far or downgraded-strategy answer — recovering most achievable quality on exactly the requests that would otherwise fail. Share-of-traffic figures are assumptions to validate in the required dry-run mode, not benchmark numbers.
- **Latency impact:** Bounds p99/p100 at the time budget by construction; ledger bookkeeping, deadline arithmetic, and per-call timeout derivation are in-memory operations adding <1 ms per hop, satisfying the ms-scale router-overhead constraint. The degradation ladder (e.g., skip refine loop -> skip escalation -> stream partial -> return cached/best-so-far) fires only near exhaustion.
- **Cost impact:** Hard per-request cost cap by construction; enables aggressive strategies (best-of-N, cascade escalation) to spend UP TO but never past the budget, so realized cost tracks budget utilization instead of worst case. Retries and hedges must debit the same ledger or the cap is fiction.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — This IS the defining capability of the project (ch. 6.4 budget-aware selection); it is pure gateway-side bookkeeping — no new infra, no GPU cost, no external dependency — and nothing else in the design (cascades, best-of-N, refine loops) is safe to ship without it.
- **Sources:**
- Google SRE Book, 'Handling Overload' (deadline propagation, retry budgets) — https://sre.google/sre-book/handling-overload/
- gRPC deadline propagation docs — https://grpc.io/docs/guides/deadlines/
- Project requirements doc AIGateway.md reqs 6.4.3, 6.5.3, 7.1.1, 7.3.1 (best-so-far and per-call timeout are stated requirements, not options)

Pre-dispatch output-length prediction (DistilBERT-scale proxy model, later self-estimation) for cost/latency feasibility estimation

- **Problem it solves:** Per-request cost (output tokens x price) and latency (TTFT + length x TPOT) are unknown until generation ends, yet req 4.1.1 demands the gateway PREDICT quality/cost/latency before routing, so a length estimate is the missing input for budget-feasibility checks, model choice, and deadline-aware scheduling.
- **Pipeline stage:** routing-decision
- **Quality impact:** Neutral on answer content; indirectly protects quality by rejecting infeasible plans before dispatch instead of truncating mid-generation. Achievable accuracy: S3's fine-tuned DistilBERT reached 98.61% length-bucket accuracy — but over 10 equal-width buckets (max-length/10 each), so the coarse bucketing does much of the work,

and the predictor was retrained online on observed mispredictions; SSJF's BERT proxy was accurate enough to drive 30.5-39.6% JCT cuts vs FCFS. Exact per-token prediction is NOT achievable — design for coarse buckets plus a 10-30% safety margin, and expect lower accuracy on the gateway's heterogeneous multi-tenant traffic than on the papers' per-dataset fine-tunes until enough in-domain logs accumulate.

- **Latency impact:** Predictor overhead ~3.7 ms per request (verified: S3 paper, DistilBERT on NVIDIA A100) — inside the ms-scale router-overhead constraint. Where length prediction has been used for scheduling: avg job completion time -30.5..-39.6% and throughput +2.2..+3.6x vs FCFS (SSJF; note FCFS is a weak baseline and these are serving-queue results on real-world datasets and production traces); +86% throughput via length-grouped batching (PiA, instruction datasets); 6.49x throughput vs worst-case KV reservation (S3 — a serving-memory-management result, not a gateway-level number). Gateway-level payoff (feasibility checks, cheapest-feasible-model choice) is an extrapolation from these serving-side results.
- **Cost impact:** No direct cost cut; enables choosing the cheapest model/strategy whose predicted cost fits the budget and refusing plans that would be paid for and then truncated. Safety margin trades away some budget headroom on over-predicted requests.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Req 4.1.1 literally mandates per-request cost/latency prediction; a DistilBERT-scale head costs ~4 ms and near-zero GPU, and can be trained on the gateway's own logged traffic (Win-Rate/TTFT logs are already required), with no external data or API dependency.
- **Sources:**
- SSJF: 'Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction' (Qiu et al., 2024) — https://arxiv.org/abs/2404.08509
- S3: 'Increasing GPU Utilization during Generative Inference for Higher Throughput' (Jin et al., NeurIPS 2023) — https://arxiv.org/abs/2306.06000
- 'Response Length Perception and Sequence Scheduling' (PiA, Zheng et al., NeurIPS 2023) — https://arxiv.org/abs/2305.13144

Budget-aware retries with exponential backoff + jitter, per-endpoint circuit breakers, and cost-gated request hedging

- **Problem it solves:** Model endpoints — especially the sanctions-unreliable closed-API fallbacks — fail, stall, or rate-limit, and naive retries either waste the request's remaining budget on a dead endpoint or synchronize into retry storms that prolong outages (req 7.4.2 mandates retry/backoff + circuit breakers).
- **Pipeline stage:** execution
- **Quality impact:** Converts endpoint failures into delivered answers: with measured transient-failure rate f and one budget-checked retry on an alternate endpoint, unanswered-rate falls from f to ~f^2 (5% -> ~0.25%, assuming independence) — arithmetic on measured rates, not a benchmark. For closed APIs whose f can spike to tens of percent, circuit breakers are the difference between burning the whole time budget probing a dead provider and an immediate local-model fallback (req 6.5.5).
- **Latency impact:** Hedged requests (duplicate to a second replica, only when cost budget allows): in The Tail at Scale's BigTable benchmark — reading 1,000 keys spread across 100 servers, hedge sent after a FIXED 10 ms delay — p99.9 fell from 1800 ms to 74 ms with ~2% extra requests; the paper's recommended production policy is to defer the hedge until the p95 expected latency, which caps added load (~5%) but yields smaller tail cuts than the 10 ms-delay figure. Caveat: that 24x tail cut is an intra-datacenter read-fan-out best case over ~100 cheap homogeneous sub-requests; hedging single high-variance LLM generation calls will show smaller, workload-dependent tail gains — the ~2-5% overhead transfers better than the magnitude. Exponential backoff + jitter empirically minimizes total work and completion time vs unjittered retries (AWS Architecture Blog simulations). Open circuit breakers cut dead-endpoint probing to ~0 ms of the remaining deadline.
- **Cost impact:** Hedging adds +2..+5% duplicate-call cost when gated on budget headroom (Tail-at-Scale-style deferred hedges; note each duplicated LLM call is far costlier than a BigTable read, so the hedge-delay threshold must be set from measured per-endpoint p95s, not copied from the paper); backoff+jitter avoids retry-storm cost amplification during incidents; circuit breakers stop paying for calls destined to time out. Every retry/hedge must debit the request's ledger.
- **Evidence label:** `evidence-based`

- **Fit for this project: high** — Req 7.4.2 is explicit, and the Iranian-market reality (closed APIs may vanish mid-request) makes per-endpoint circuit breakers + instant local fallback existential rather than nice-to-have; it is cheap gateway-core code, and archgw/Envoy heritage gives outlier-detection primitives nearly for free.
- **Sources:**
- Dean & Barroso, 'The Tail at Scale', CACM 2013 — https://cacm.acm.org/research/the-tail-at-scale/ (benchmark: 1,000 keys across 100 BigTable servers, 10 ms hedge delay)
- AWS Architecture Blog, 'Exponential Backoff and Jitter' — https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Google SRE Book, 'Handling Overload' (retry budgets, ~10% retry cap) — https://sre.google/sre-book/handling-overload/
- Envoy outlier detection (per-endpoint ejection) — https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/outlier

Health-, load-, and prefix-cache-aware load balancing across vLLM replicas + rate-limit-aware provider dispatch + KubeAI autoscaling/scale-to-zero

- **Problem it solves:** Which replica or provider serves a request determines TTFT, tail latency, and KV-cache reuse; naive round-robin scatters shared prefixes across replicas, keeps routing to slow/dead endpoints (violating req 6.2.3), trips closed-API 429s, and a statically sized GPU fleet pays peak cost all day.
- **Pipeline stage:** serving-layer
- **Quality impact:** neutral
- **Latency impact:** Cache-aware LB: up to 1.9x throughput (82,665 -> 158,596 tok/s) and 3.8x higher prefix-cache hit rate (20% -> 75%) vs round-robin — verified against the SGLang v0.4 blog, BUT on a synthetic workload of multiple long-prefix groups with each group perfectly balanced, on 8x A100; treat as an upper bound, with production gains scaling down with the real shared-prefix fraction and prefix-group balance of traffic. On system-prompt-heavy traffic hits skip prefill, so TTFT cuts can still be large. Least-loaded + health-aware routing removes slow/dead replicas from rotation. Caveat: autoscaling scale-up is minutes-scale (vLLM pod must load weights), so it must be padded/predictive or tight time budgets will miss during ramps.
- **Cost impact:** 1.4-1.9x throughput per GPU on shared-prefix traffic = ~25-45% fewer GPU-hours for the same load (derived from the SGLang best-case synthetic benchmark — an upper bound, not an expectation; measure on own traffic before crediting savings); KubeAI load-based autoscaling + scale-to-zero for rarely used models saves GPU-hours roughly proportional to traffic diurnality (~30-60% if off-peak load is ~3x lower — reasoned estimate, no published benchmark); tracking provider rate-limit windows avoids 429-triggered retry waste on the closed-API tier.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — The stack is already KubeAI + vLLM on K8s, so prefix-aware LB and autoscaling are configuration-level wins, and the gateway must track per-endpoint health/latency anyway (req 6.2.3); with near-zero budget, cache-aware LB is the cheapest GPU-capacity multiplier available. Note the autoscaling savings figure is reasoned (not benchmarked) and the LB figures are a synthetic-workload upper bound.
- **Sources:**
- SGLang v0.4 cache-aware load balancer benchmark — https://lmsys.org/blog/2024-12-04-sglang-v0-4/ (synthetic balanced long-prefix workload, 8x A100)
- vLLM Router (Rust-based prefill/decode- and cache-aware LB; source verified) — https://vllm.ai/blog/2025-12-13-vllm-router-release
- KubeAI docs: load balancing (PrefixHash) and autoscaling — https://www.kubeai.org/
- LiteLLM router (usage/rate-limit-based provider routing, practice reference) — https://docs.litellm.ai/docs/routing

Priority/SLO-aware inference scheduling (remaining-deadline -> vLLM priority mapping; QLM-style queue management later)

- **Problem it solves:** FCFS continuous batching suffers head-of-line blocking: long low-urgency jobs sit in front of tight-deadline interactive requests, so requests miss their time budgets even when total capacity is sufficient.
- **Pipeline stage:** serving-layer

- **Quality impact:** Neutral on answer content; raises the fraction of requests finishing inside their time budget — i.e., fewer degraded best-so-far returns. QLM improved SLO attainment by 40-90% vs vanilla vLLM (verified; also 50-90% vs SHEPHERD) on mixed interactive+batch workloads (SoCC 2024).
- **Latency impact:** QLM: +40-90% SLO attainment and +20% average single-model throughput vs vanilla vLLM and EDF (verified; +400% average throughput in multi-model serving, driven by request-group scheduling that minimizes model swaps plus model warm start — the ablation credits warm start alone with +300% — so the big number only applies if the fleet actually swaps many models per GPU); SSJF-style shortest-predicted-job-first ordering: avg JCT -30.5..-39.6%, throughput +2.2..3.6x vs FCFS (FCFS baseline — gains over a well-tuned scheduler will be smaller). The cheap slice — gateway maps remaining time budget to vLLM's built-in priority scheduling (--scheduling-policy priority) so tight-deadline requests preempt batch traffic — has sub-ms decision overhead.
- **Cost impact:** Neutral-to-positive: higher SLO attainment at fixed capacity defers GPU purchases; preemption wastes some recomputed decode tokens (bounded, single-digit % of tokens at moderate preemption rates — estimate, workload-dependent).
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — The team runs its own vLLM, so deadline->priority mapping is an integration task with built-in scheduler support and directly serves the budget contract; the full QLM-style global queue manager (stochastic programming, model swapping) is Tier-3 for a 6-person team — adopt only if priority mapping measurably leaves SLO attainment on the table.
- **Sources:**
- QLM: 'Queue Management for SLO-Oriented Large Language Model Serving' (Patke et al., SoCC 2024) — https://arxiv.org/abs/2407.00047
- SSJF (Qiu et al., 2024) — https://arxiv.org/abs/2404.08509
- vLLM priority scheduling docs (scheduling-policy=priority) — https://docs.vllm.ai/

Speculative decoding at the serving layer (vLLM-owned: n-gram/draft-model/EAGLE-3; gateway routes to it, does not implement it)

- **Problem it solves:** Autoregressive decode is memory-bandwidth-bound, so TPOT dominates end-to-end latency for long outputs and makes tight time budgets hard to meet even on idle H100s.
- **Pipeline stage:** serving-layer
- **Quality impact:** Neutral (0% change) by construction: rejection sampling guarantees the output distribution is identical to the target model's (Leviathan et al.); verify per vLLM mode, since some acceptance-relaxation settings trade this guarantee away.
- **Latency impact:** 2-3x faster decoding (Leviathan et al., ICML 2023, T5-XXL, batch-1, output-identical); vLLM measured up to 2.8x lower latency with draft-model/n-gram speculation at low QPS (ShareGPT/CNN-DM), and up to ~2.5x with EAGLE-3 (EAGLE-3 paper claims up to 6.5x at batch-1 — vendor-measured vLLM gains are the transferable figure, not the paper's peak). Critical condition: speedup decays toward ~1x (and can go negative for throughput) as batch size/QPS saturate the GPU — this is a latency-tier lever, not a fleet-wide throughput lever.
- **Cost impact:** At low concurrency, faster completion frees GPU-seconds: roughly -20..-50% GPU-time per request (and energy, since decode power draw scales with time per the project's edge-inference brief); at saturation, draft work competes with batch throughput, so cost impact is ~0 or negative — enable per-endpoint, not globally.
- **Evidence label:** `evidence-based`
- **Fit for this project: medium** — It is a vLLM configuration owned by the serving side, not gateway code; the gateway's job is only to know which endpoints have it enabled and route tight-time-budget requests there — worth enabling on a dedicated latency-tier deployment, but scarce H100 capacity running near saturation sees little benefit.
- **Sources:**
- Leviathan et al., 'Fast Inference from Transformers via Speculative Decoding', ICML 2023 — https://arxiv.org/abs/2211.17192
- vLLM blog, 'How Speculative Decoding Boosts vLLM Performance by up to 2.8x' — https://blog.vllm.ai/2024/10/17/spec-decode.html

- Red Hat Developer / vLLM EAGLE-3 posts — https://developers.redhat.com/articles/2025/07/01/fly-eagle3-fly-faster-inference-vllm-speculative-decoding

Disaggregated prefill/decode serving (DistServe/Splitwise/Mooncake pattern; llm-d / vLLM + NIXL implementations)

- **Problem it solves:** Colocated prefill and decode interfere in continuous batching — long prefills stall in-flight decodes and vice versa — inflating TTFT/TPOT tails and forcing over-provisioning to meet both SLOs simultaneously.
- **Pipeline stage:** serving-layer
- **Quality impact:** neutral
- **Latency impact:** DistServe: 7.4x more requests served within tight TTFT+TPOT SLOs, or 12.6x tighter SLOs, at >90% SLO attainment (OSDI 2024, OPT-13B..175B, A100 testbed); Splitwise: 1.4x throughput at 20% lower cost, or 2.35x throughput at iso-cost/power (ISCA 2024, production Azure traces); Mooncake: up to 5.25x throughput in simulated scenarios and +75% real requests handled (Kimi production, KV-centric scheduling). All are paper-author-measured figures on their own testbeds/simulators — treat as headline best cases.
- **Cost impact:** -20% cost at 1.4x throughput (Splitwise) — but only once the fleet is large enough to dedicate separate prefill and decode pools with fast KV transfer (RDMA/NVLink/NIXL); below roughly a two-digit GPU count, the pool split strands capacity and the KV-transfer fabric adds engineering cost that outweighs gains.
- **Evidence label:** `evidence-based`
- **Fit for this project: low** — Multi-node KV-transfer infrastructure is premature for a 6-person team with a small H100 fleet and an 8-month roadmap; the actionable item is keeping the gateway's endpoint abstraction P/D-agnostic (route to a logical endpoint, not a physical pod) so the serving side can adopt disaggregation later with zero gateway changes.
- **Sources:**
- DistServe (Zhong et al., OSDI 2024) — https://arxiv.org/abs/2401.09670
- Splitwise (Patel et al., ISCA 2024) — https://arxiv.org/abs/2311.18677
- Mooncake (Qin et al., Moonshot AI) — https://arxiv.org/abs/2407.00079

Quantization tiers — AWQ INT4 (W4A16) and FP8 (W8A8) model variants as the cheap-tier capacity backbone

- **Problem it solves:** The router can only trade cost for quality if a genuinely cheaper tier exists; on self-owned H100s 'cheaper' means more tokens per GPU-hour, which quantized variants of the same open-weight models provide at near-identical quality.
- **Pipeline stage:** offline
- **Quality impact:** -0..-3% task accuracy depending on format (verified against Kurtic et al. abstract): FP8 W8A8 'effectively lossless across all model scales', well-tuned INT8 W8A8 1-3% degradation, INT4 W4A16 ~99% average score recovery ('rivaling 8-bit quantization') — across academic and real-world benchmarks (Arena-Hard, OpenLLM) on the full Llama-3.1 family, 500k+ evaluations. Worst cases concentrate in small models and long-form reasoning; every quantized artifact must be re-validated on the promptfoo golden set before joining a tier.
- **Latency impact:** AWQ INT4: the paper's >3x speedup vs HuggingFace FP16 is TinyChat on desktop/mobile GPUs (RTX 4090/Jetson Orin class) against an unoptimized HF baseline — expect smaller (though still real) low-batch gains on H100 + vLLM via Marlin kernels; FP8 on H100: up to 2x latency reduction and ~1.6x throughput in vLLM (Neural Magic/Red Hat vendor measurements). Deployment guidance from the 500k-eval study (verified): 'W4A16 is the most cost-efficient for synchronous setups, while W8A8 dominates in asynchronous continuous batching' on H100.
- **Cost impact:** ~4x weight-memory reduction (INT4) / 2x (FP8) lets bigger models fit per H100 or more replicas per node; tokens per GPU-hour up 1.6-3x, so cheap-tier serving cost falls roughly -40..-70% vs BF16 serving of the same model — a derived estimate, conditions: H100-class GPUs, vLLM AWQ/Marlin/FP8 kernels, quality re-verified per model.
- **Evidence label:** `evidence-based`

- **Fit for this project: high** — This manufactures the price-quality gradient the whole gateway monetizes, on hardware the team already owns, as one-time offline work per model with native vLLM support — for a near-zero-budget, sanctions-constrained self-hosting operation it is the single cheapest way to create a real cheap tier.
- **Sources:**
- AWQ (Lin et al., MLSys 2024 Best Paper) — https://arxiv.org/abs/2306.00978 (3x figure is TinyChat on desktop/mobile GPUs)
- 'Give Me BF16 or Give Me Death? Accuracy-Performance Trade-Offs in LLM Quantization' (Kurtic et al., ACL 2025) — https://arxiv.org/abs/2411.02355
- Red Hat Developer, 'vLLM brings FP8 inference to the open source community' — https://developers.redhat.com/articles/2024/07/15/vllm-brings-fp8-inference-open-source-community

Fact-check verdicts for this area

Claim	Verdict	Note
S3: fine-tuned DistilBERT length predictor reaches 98.6% bucket accuracy, costs ~3.7 ms per request on A100, and enables 6.49x throughput vs worst-case KV reser	**confirmed**	Full text states 'this predictor predicts the correct bucket with 98.61% accuracy' and 'Distilbert takes 3.7ms to run... on an NVIDIA A100 GPU'; abstract states '6.49x throughput over those systems that assume the worst case'. Nuance: accuracy is over 10 equal-width buckets (max_len/10 each), so coa
SSJF: average job completion time -30.5..-39.6% and throughput +2.2-3.6x vs FCFS	**confirmed**	Abstract matches verbatim: 'reduces average job completion times by 30.5-39.6% and increases throughput by 2.2-3.6x compared to FCFS schedulers' across no-batching/dynamic/continuous-batching settings. Caveat: FCFS is a weak baseline and these are serving-queue scheduling gains; the findings correct
Hedged requests cut p99.9 from 1800 ms to 74 ms with ~2% extra requests in a 'BigTable 1000-server read benchmark', hedging after ~p95 delay	**corrected**	The 1800ms->74ms p99.9 and ~2% extra-request numbers are genuine, but the setup was misattributed twice: the benchmark reads 1,000 KEYS spread across 100 servers (not 1000 servers), and that specific measurement used a FIXED 10 ms hedge delay — deferring until p95 latency is the paper's general over
SGLang v0.4 cache-aware load balancer: up to 1.9x throughput and 3.8x higher prefix-cache hit rate vs round-robin	**confirmed**	Blog confirms 1.9x (82,665 -> 158,596 tok/s) and 3.8x hit rate (20% -> 75%) on 8x A100 — but the workload is synthetic: 'multiple long prefix groups, and each group is perfectly balanced'. This is a best-case upper bound; production gains scale with the actual shared-prefix fraction. Caveat added in
QLM: SLO attainment +40-90% vs vanilla vLLM; throughput +20% single-model and +400% multi-model	**confirmed**	Full text: 'QLM achieves 40-90% higher SLO attainment compared to the vanilla vLLM serving system' and 'improves the request throughput in a multi-model serving system by 400% on average and in a single-model serving system by 20% on average'. Refinement: the multi-model gain is driven by request-gr
Kurtic et al.: FP8 W8A8 effectively lossless, well-tuned INT8 -1..-3%, W4A16 ~99% recovery, 500k+ evaluations on the Llama-3.1 family; W4A16 best for synchronou	**confirmed**	Abstract confirms all: 'FP8 (W8A8-FP) is effectively lossless across all model scales', 'well-tuned INT8 (W8A8-INT) achieves surprisingly low (1-3%) accuracy degradation', W4A16 'rivaling 8-bit quantization', 'over 500,000 evaluations', 'entire Llama-3.1 model family', and 'W4A16 is the most cost-ef

Quality measurement: judges, reward models, eval harness, XAI

Self-hosted generative LLM-as-judge (7-8B, Prometheus-2-class or Qwen-instruct, pairwise + rubric prompts)

- **Problem it solves:** Scores subjective answer quality (helpfulness, faithfulness) without human labels and without depending on sanctions-unreliable closed judge APIs.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** neutral direct (measurement infrastructure); measurement validity: GPT-4-class judges reach over 80% agreement with humans on MT-Bench chat traffic (85% vs human experts in the paper body; human-human agreement is ~81%). Open Prometheus-2-7B reaches Pearson 0.882 vs GPT-4 scores on Feedback Bench — but that is its IN-DOMAIN test set (generated and scored by the same GPT-4-0613 as its training data); on out-of-domain direct-assessment benchmarks it drops to Pearson 0.54-0.69 vs GPT-4-1106, and its human correlation is measured only on FLASK at ~0.545, marginally above 0.5 — treat 0.88 as a ceiling, 0.55-0.65 as the realistic out-of-distribution expectation. Valid ONLY with position-swapped pairwise prompting and length/verbosity controls (position and

verbosity biases are documented in the MT-Bench paper); agreement degrades further on specialized technical/Persian-domain content, so spot-check against human labels on the golden set.

- **Latency impact:** 0% when run async off the critical path (recommended default); +200-1500ms per judged request if placed in-path as a cascade scorer (one 7-8B generation) — prefer the reward-model scorer for in-path use.
- **Cost impact:** ~$0.0001-0.0005 per judgment self-hosted on existing H100s (throughput arithmetic at $2-3/GPU-hr for an 8B model) vs ~$0.01-0.03 per judgment on GPT-4-class APIs, i.e. 20-100x cheaper; judging a 1-5% traffic sample adds ~1-5% total inference cost.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Closed APIs are explicitly unreliable in this project's market, so a co-hosted open judge on the existing vLLM/H100 stack is the only dependable way to score subjective quality; Qwen-family weights are already being served.
- **Sources:**
- Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (Zheng et al., NeurIPS 2023) — https://arxiv.org/abs/2306.05685
- Prometheus 2: An Open Source LM Specialized in Evaluating Other LMs (Kim et al. 2024) — https://arxiv.org/abs/2405.01535 (Table 3: Feedback Bench 0.882/0.898 in-domain; out-of-domain 0.54-0.69 vs GPT-4-1106; FLASK-human 0.545/0.555)

Small sequence-classifier reward model as cascade scorer / best-of-N selector (Skywork-Reward-V2 0.6B-8B, ArmoRM-8B scale)

- **Problem it solves:** Provides a single-forward-pass 'is this answer good enough' score for cascade escalation (FrugalGPT scorer role) and best-of-N candidate selection, at a fraction of a generative judge's cost and latency.
- **Pipeline stage:** validation-synthesis
- **Quality impact:** neutral direct; as the cascade scorer it is the component that determines whether the cascade holds quality: FrugalGPT reports matching GPT-4 quality at up to -98% cost, but that is the single best case (98.3% on HEADLINES, a narrow news-classification task; its other two tasks show 73.3% and 59.2%, and all three are closed-form classification/QA, not open-ended chat — expect materially less on mixed production traffic; RouteLLM-adjacent results suggest 35-85% depending on traffic mix) using a DistilBERT regression scorer. Scorer accuracy: current top 8B RMs reach ~97-98 on RewardBench v1 (Skywork-Reward-V2-Llama-3.1-8B-40M: 97.8 — v1 is effectively saturated; the harder RewardBench v2 tops out at 86.5, the more honest number), and Skywork-Reward-V2-Qwen3-0.6B nearly matches the prior-generation 27B RM on average across 7 RM benchmarks (70.9 vs 71.6). Caveat: RM scores are relative preferences, not calibrated probabilities — escalation thresholds must be tuned per task family against the golden set, or the cascade either escalates everything (no savings) or passes garbage (silent regression).
- **Latency impact:** +10-50ms per scored answer (one prefill pass of a 0.6B-8B classifier on H100, no decoding); it runs post-generation, so the ms-scale router-overhead requirement (pre-decision) is untouched.
- **Cost impact:** ~1-3% of the cost of the generation it scores (single forward pass over prompt+answer); self-hosted on shared GPU capacity, zero API spend.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — The natural Phase-2 cascade scorer: open weights, the 0.6-1.7B variants fit beside serving models on existing GPUs (vLLM serves classifier/pooling-style models), and it needs no labeled training data — only threshold calibration on the golden set.
- **Sources:**
- FrugalGPT (Chen, Zaharia, Zou 2023) — https://arxiv.org/abs/2305.05176 (Table 3: cost savings 98.3% HEADLINES / 73.3% OVERRULING / 59.2% COQA; 'We employ a DistilBERT tailored to regression as the scoring function')
- Skywork-Reward-V2: Scaling Preference Data Curation via Human-AI Synergy — https://arxiv.org/abs/2507.01352
- Skywork-Reward-V2-Qwen3-0.6B model card — https://huggingface.co/Skywork/Skywork-Reward-V2-Qwen3-0.6B (0.6B avg 70.9 vs 27B-v0.2 avg 71.6 over 7 benchmarks; 8B-40M: RewardBench v1 97.8, v2 86.5)
- RewardBench (Lambert et al. 2024) — https://arxiv.org/abs/2403.13787

promptfoo golden-set eval harness (deterministic assertions first, judge assertions sparingly)

- **Problem it solves:** Proves that every routing-threshold, model-swap, or scorer change held quality, cost and latency on a fixed representative request set before it ships.
- **Pipeline stage:** offline
- **Quality impact:** neutral direct; protective: a 200-500-item labeled golden set reliably detects >=5pp pass-rate regressions (binomial noise ~2.3pp at n=300; grow toward 1000+ items to detect 2-3pp); the strategy brief's phase gates ('<5% quality regression at >40% cost savings') are unverifiable without it. Deterministic assertions (regex, contains, valid-JSON, schema, latency<X, cost<Y) are noise-free; judge assertions inherit the judge's ~80-85% human-agreement ceiling — use them only for genuinely subjective checks.
- **Latency impact:** 0% at runtime (CI/offline only).
- **Cost impact:** per full run: minutes of self-hosted GPU time (~$0-5 equivalent); deterministic assertions are token-free; promptfoo itself is free and open-source and points at vLLM endpoints directly.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Mandated day one by the strategy brief; zero budget, model-agnostic, and it is the instrument every other technique's threshold (cascade, cache, router) is tuned against.
- **Sources:**
- promptfoo assertion docs — https://www.promptfoo.dev/docs/configuration/expected-outputs/
- Project strategy brief AIGateway__4_.md §4 (mandates it day one; detection floor derived from sample-size statistics)

Online quality telemetry + sampled Win Rate vs strong tier (req 4.1.5)

- **Problem it solves:** Continuously measures the live-traffic quality/cost trade-off so routing and cascade thresholds can be tuned against real distribution shift, not just the static golden set.
- **Pipeline stage:** serving-layer
- **Quality impact:** neutral direct; enabling: win-rate-vs-strong-model is the exact metric under which RouteLLM demonstrates over 85% cost reduction while retaining 95% of GPT-4-level performance — but that figure is its best-case benchmark (MT-Bench chat traffic); on MMLU the saving falls to 45% and on GSM8K to 35% at the same 95% quality retention, so expect the mid-range, not 85%, on mixed production traffic. The project's own Win Rate KPI (req 4.1.5) reuses this methodology. Valid only if the judge sample is position-swapped and the reference (strong tier) is held fixed across comparison windows; hard metrics (cost, TTFT, router overhead, cache-hit, budget-met) are logged on 100% of requests.
- **Latency impact:** 0% (hard-metric logging is in-line microseconds on the existing Prometheus stack; reference generation + judging is fully async off-path).
- **Cost impact:** +2-6% total inference cost at a 1-2% sampling rate (each sampled request pays one strong-tier reference generation plus one self-hosted judgment); tune the sampling knob to budget.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Explicit hard requirement (4.1.5); it reuses infrastructure the project already must build (per-request structured logs, Prometheus metrics, self-hosted judge) with only a sampling worker added.
- **Sources:**
- RouteLLM: Learning to Route LLMs with Preference Data (Ong et al. 2024) — https://arxiv.org/abs/2406.18665 (85% MT-Bench / 45% MMLU / 35% GSM8K cost reduction at 95% of GPT-4 performance)
- RouteLLM release post — https://lmsys.org/blog/2024-07-01-routellm/
- Project requirements doc AIGateway.md req 4.1.5, 7.6.4

Dry-run counterfactual + shadow/replay policy evaluation (req 1.1.4)

- **Problem it solves:** Evaluates a candidate routing policy on real traffic without user-facing risk and — in dry-run/replay modes — without paying model inference cost.
- **Pipeline stage:** routing-decision
- **Quality impact:** neutral direct; de-risking: (a) decision-only dry-run logs the counterfactual model choice, giving EXACT cost/latency deltas at zero model spend but only PREDICTED quality; (b) an executed-shadow sample (call

the shadow-chosen model, judge async) yields the true quality delta; (c) RouterBench-style offline replay against precomputed outputs (405,467 inference outcomes from 11 LLMs across 8 datasets in the benchmark; locally, precompute golden-set outputs for every hosted model once) evaluates unlimited router variants at zero marginal inference cost.

- **Latency impact:** +<1-5ms if the shadow policy is evaluated in-line next to the live router decision (a second ms-scale policy pass); 0 if replayed offline from the per-request logs.
- **Cost impact:** decision-only dry-run: ~0; executed shadow on s% of traffic: +s% x shadow-model cost (e.g. +2-5% at 2-5% sampling); offline replay: one-time precompute of golden-set outputs per model, then free forever.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Dry-run mode is a hard requirement, and log-replay converts the already-required per-request logging into a free, growing router-evaluation dataset — the cheapest possible path to safe policy iteration for a 6-person team.
- **Sources:**
- RouterBench: A Benchmark for Multi-LLM Routing Systems (Hu et al. 2024) — https://arxiv.org/abs/2403.12031 (abstract: 'over 405k inference outcomes'; 11-model/8-dataset breakdown per dataset docs and https://withmartian.com/post/introducing-routerbench)
- Project requirements doc AIGateway.md req 1.1.4 (dry-run mode)

Silent-regression protection: CI quality gate + judge-scored canary with auto-rollback

- **Problem it solves:** Catches the 'cheaper but quietly worse' failure mode — the strategy brief's #1 named risk — before and during rollout instead of after user churn.
- **Pipeline stage:** serving-layer
- **Quality impact:** bounds any shipped regression to the detection floor instead of leaving it unbounded: ~5pp on a 300-item CI golden-set gate, ~2-3pp with a 1000+-judgment canary window during blue-green rollout (req 2.1.2); cheap drift alarms on escalation rate, mean RM score, and refusal/length distributions catch shift between releases at zero judge cost. Conditions: floors follow binomial statistics, so small canaries cannot see small regressions — size the window to the regression you must catch.
- **Latency impact:** 0% (CI gate is offline; canary judging is async; blue-green switchover is already required infrastructure).
- **Cost impact:** +1-3% during canary windows only (async judging of the canary sample); zero steady-state cost.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: high** — Composes three things the project must build anyway (golden set, self-hosted judge, blue-green deploys) into the guarantee that makes the 'best answer within budget' promise credible; no new components.
- **Sources:**
- Detection floors derived from sample-size statistics; practice pattern extrapolated from standard canary/SRE deployment literature and promptfoo CI usage — https://www.promptfoo.dev/docs/integrations/ci-cd/
- Project strategy brief AIGateway__4_.md §4, §6 (names silent quality regression the #1 failure mode)

Template-first XAI routing explanations (decision features -> NL template; optional cached LLM polish)

- **Problem it solves:** Produces the required per-decision human-readable routing explanation (req 4.1.4) without adding latency, token cost, or fabricated rationales.
- **Pipeline stage:** routing-decision
- **Quality impact:** neutral on answer quality; explanation faithfulness: templates rendered from the actual decision features (intent class, per-candidate predicted quality/cost/latency, budget fit, cache/cascade/fallback events) are faithful by construction, whereas free-form LLM-generated rationales are documented to plausibly misstate the true cause of a decision (unfaithful post-hoc explanations, Turpin et al.) — a real audit risk for an XAI requirement.
- **Latency impact:** template: <1ms in-path (string formatting over fields the router already computed), compatible with the ms-scale router-overhead requirement (4.2.1); per-request LLM generation: +100-800ms if in-path — only acceptable async or pre-cached per (intent, model, budget-band) tuple.

- **Cost impact:** template: ~0; per-request LLM rationale: one extra small-model generation per request, roughly +3-10% of a cheap-tier call — avoid; hybrid (LLM-polished phrasing generated once per routing-policy cell, then cached) amortizes to ~0.
- **Evidence label:** `evidence-based`
- **Fit for this project: high** — Satisfies the XAI requirement at zero budget and zero overhead using features the router already computes; a per-request LLM explanation would violate the ms-overhead constraint and is less trustworthy, not more.
- **Sources:**
- Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting (Turpin et al., NeurIPS 2023) — https://arxiv.org/abs/2305.04388
- Project requirements doc AIGateway.md req 4.1.4, 4.2.1

End-user feedback capture (thumbs/regenerate/edit signals) as free online quality labels

- **Problem it solves:** Grounds judge and reward-model proxies in real user preference and accumulates the labeled preference dataset a Phase-4 trained router would need.
- **Pipeline stage:** serving-layer
- **Quality impact:** neutral direct; long-run enabler: preference-data-trained routers (RouteLLM-style, trained on exactly this kind of signal) achieve 35-85% cost reduction at ~95% quality retention (85% is the MT-Bench chat best case; RouteLLM's own MMLU and GSM8K results are 45% and 35%) — but explicit feedback is sparse (typically only a low-single-digit % of requests) and biased toward failures, so treat it as calibration for the judge/RM, not as a primary metric.
- **Latency impact:** 0% (optional out-of-band feedback endpoint keyed on the per-request correlation ID from req 6.7.2).
- **Cost impact:** ~0 (an API field plus storage); replaces future paid labeling for router training data.
- **Evidence label:** `reasoned-estimate`
- **Fit for this project: medium** — Nearly free to add, but the gateway exposes an OpenAI-compatible API, so feedback volume depends on client applications cooperating — useful calibration signal, unreliable as sole quality telemetry.
- **Sources:**
- RouteLLM (Ong et al. 2024, trained on Chatbot Arena human preference data) — https://arxiv.org/abs/2406.18665; per-benchmark savings 85%/45%/35% verified against https://lmsys.org/blog/2024-07-01-routellm/; feedback-rate sparsity extrapolated from product-analytics norms, no LLM-specific benchmark

Fact-check verdicts for this area

Claim	Verdict	Note
GPT-4-class judges reach 80-85% agreement with humans on MT-Bench chat traffic; human-human agreement is ~81%; position and verbosity biases documented	**confirmed**	Abstract states GPT-4 achieves 'over 80% agreement, the same level of agreement between humans' on both controlled and crowdsourced human preferences, and explicitly documents position, verbosity, and self-enhancement biases. The 85% / 81% specifics come from the paper body (GPT-4 vs human experts 8
Prometheus-2-7B reaches Pearson ~0.88 vs GPT-4 scores on Feedback Bench and >0.5 vs humans across direct-assessment benchmarks	**corrected**	The 0.88 is real (0.882 for 7B, 0.898 for 8x7B vs GPT-4-0613) but it is the IN-DOMAIN number: Feedback Bench is Prometheus's own training distribution, scored by the same GPT-4-0613 that generated the training data. Out-of-domain correlations vs GPT-4-1106 are 0.54-0.69 (Vicuna Bench 0.642, MT-Bench
FrugalGPT matches GPT-4 quality at up to -98% cost (HEADLINES, best case) using a DistilBERT-scale scorer	**confirmed**	Verified verbatim: 98.3% cost savings on HEADLINES, and 'We employ a DistilBERT tailored to regression as the scoring function.' However the other two tasks show much smaller savings (OVERRULING 73.3%, COQA 59.2%) and all three are narrow classification/QA tasks, not open-ended chat — the analyst al
Top 8B RMs score ~93-95 on RewardBench v1, and Skywork-Reward-V2-Qwen3-0.6B matches a prior-generation 27B RM on average across 7 RM benchmarks	**corrected**	Two corrections. (1) '~93-95' is a stale prior-generation figure: Skywork-Reward-V2-Llama-3.1-8B-40M scores 97.8 on RewardBench v1 — v1 is effectively saturated — while the harder RewardBench v2 tops out at 86.5 for the same model, which is the more honest headroom number. (2) The 0.6B model 'nearly

Claim	Verdict	Note
RouteLLM demonstrates up to ~85% cost reduction while retaining ~95% of strong-model performance on MT-Bench-like chat traffic (win-rate-vs-strong-model methodo	**confirmed**	Verified: 'over 85%' cost savings on MT-Bench, 45% on MMLU, 35% on GSM8K vs GPT-4-only, at 95% of GPT-4 performance. MT-Bench is explicitly the best of the three benchmarks — the 45%/35% figures for knowledge/math traffic are now written into the impact text as the transfer caveat. Note the arXiv ab
RouterBench provides 405k responses from 11 LLMs for offline replay	**confirmed**	Abstract confirms 'over 405k inference outcomes' (405,467 samples); the 11-model / 8-dataset / 64-task breakdown is confirmed by the dataset documentation and Martian's release post rather than the abstract itself. Numbers match as cited.